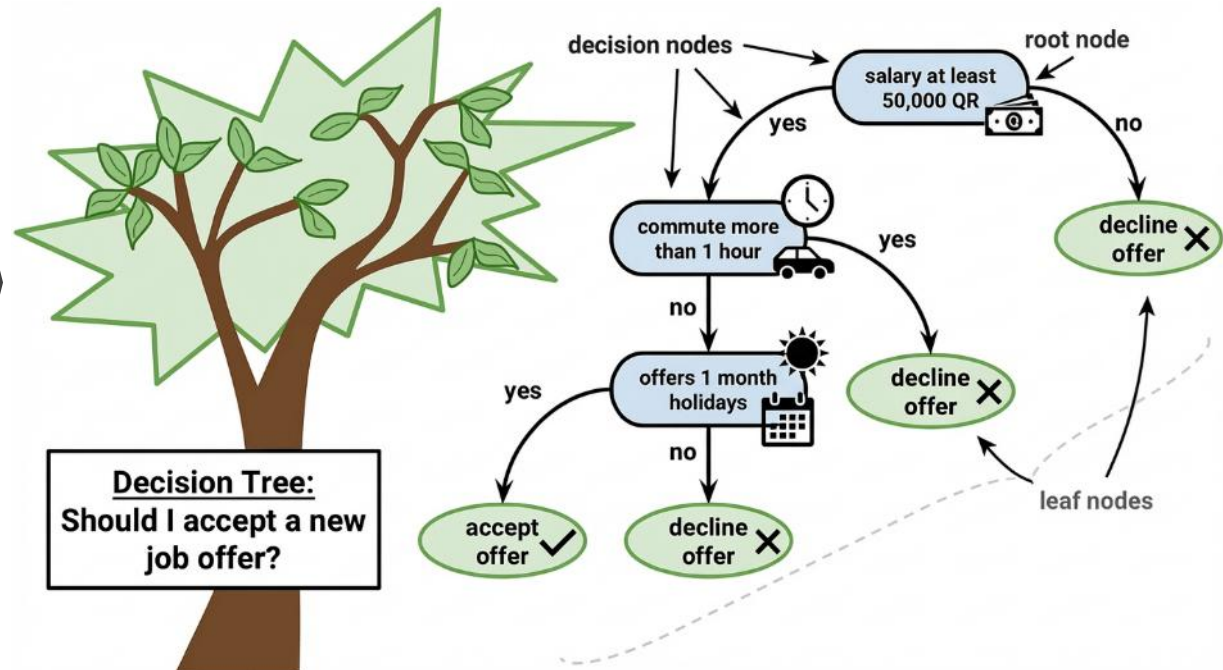


Classification using Decision Trees

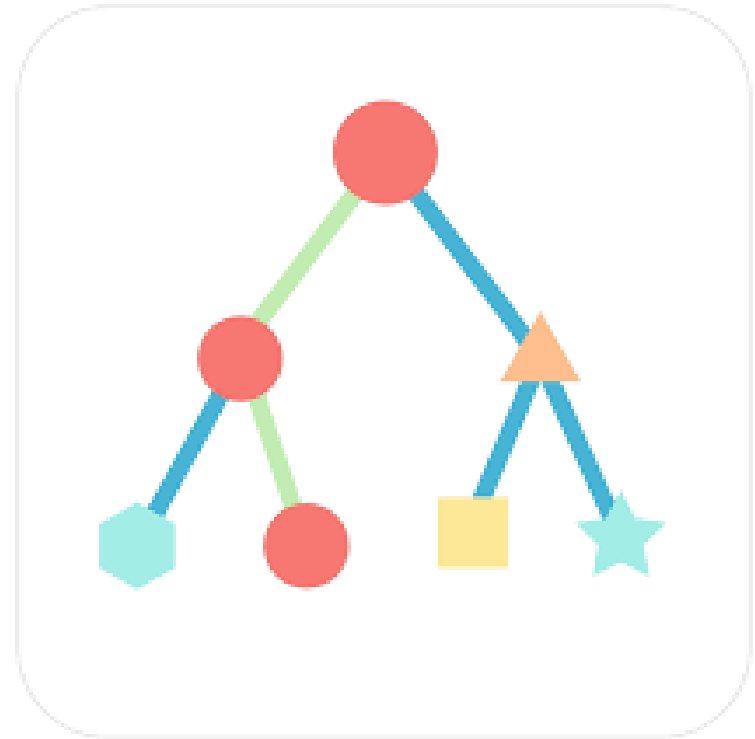


Outline

- Decision Tree (DT)
- **DT Attribute Selection Measure (ASM)**
 - Classification Error Rate
 - Gini Impurity Index
 - Entropy
- Decision Boundaries



Decision Trees



Should I wear jacket today?

- 🌧️ Raining → Wear a jacket 🧥
- ☀️ Not raining → Check temperature
 - 🔥 Hot → Don't wear a jacket
 - 🥶 Cold → Wear a jacket 🧥

Key ideas of a Decision Tree (DT):

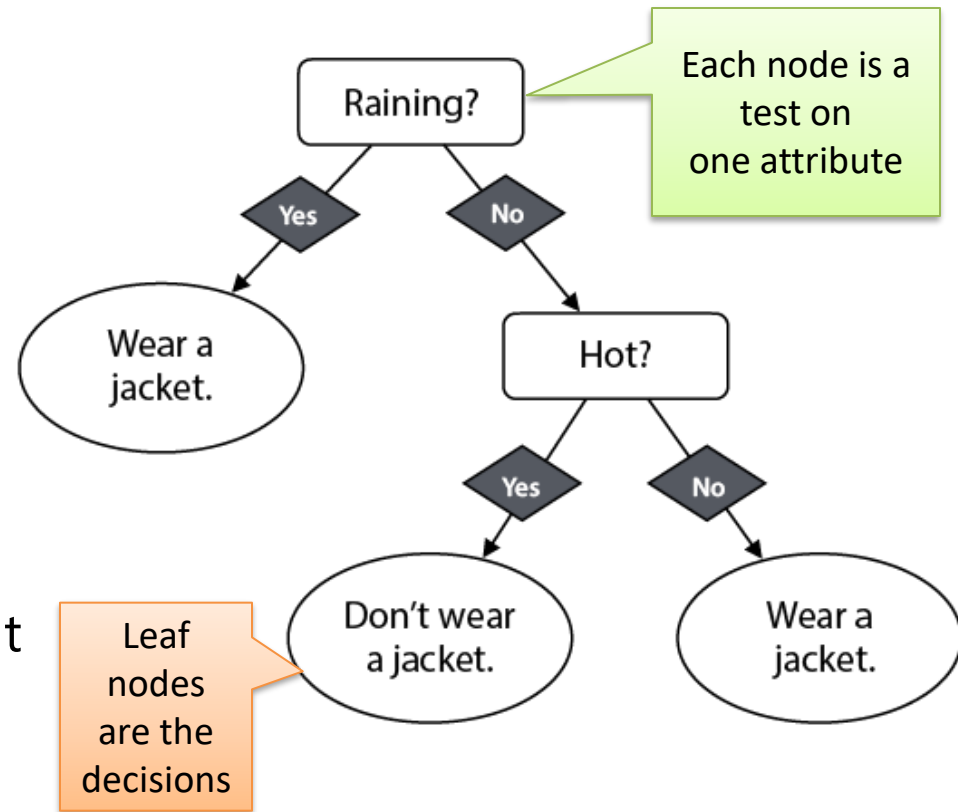
🌳 **DT:** depicts the decision process

❓ **Each node:** asks a question about one attribute

🌿 **Each branch:** outcome of the question (yes or no)

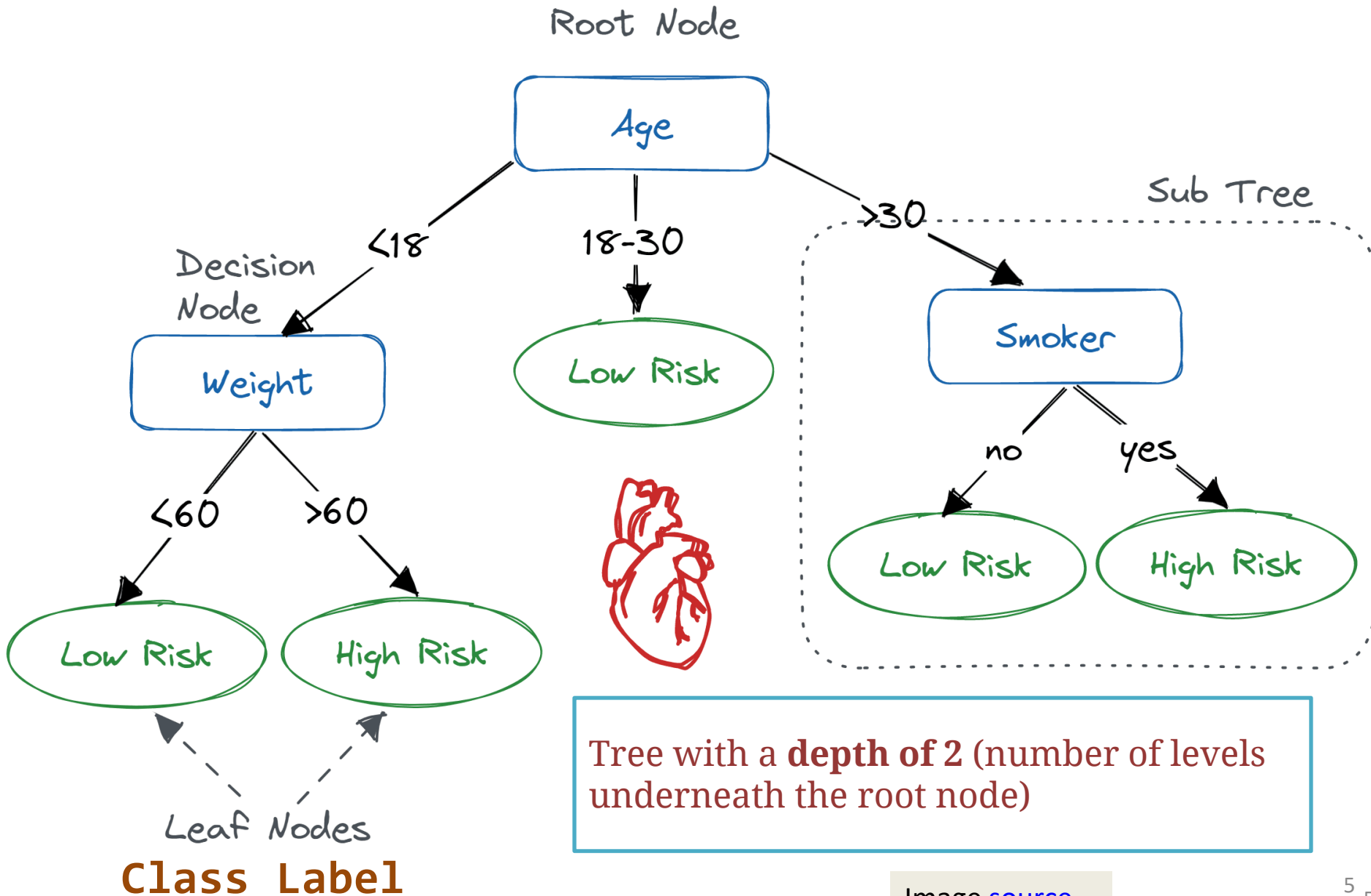
🎯 **Each leaf:** final decision/class

⬇️ **Flow:** top → bottom until reaching the decision



We arrive to a decision (i.e., a class) by asking a series of questions

Decision Tree – Risk of heart attack



Decision Tree (DT)

🌳 **DT:** is a supervised ML model that uses a sequence of yes-or-no questions to classify instances

- **Key components:**



Root node: starting point of the model



Decision nodes: test a feature



Branches: outcomes of tests



Leaf nodes: final class prediction

When to use DTs:



When transparent **rule-based decisions** are required



When decisions depend on **conditional checks** across attributes



When **interpretability** and **explainability** are essential (e.g., loan approval, fraud detection, medical triage)

Decision Tree for the Iris dataset: using all the four attributes

```
X_iris = iris.data.values
y_iris = iris.target
tree_clf = DecisionTreeClassifier(max_depth=4, random_state=42)
tree_clf = tree_clf.fit(X_iris, y_iris)
```

```
tree_clf.fit(X_iris, y_iris)
plt.figure(figsize=(10,8))
plot_tree(tree_clf, filled=True)
plt.title("Decision tree trained on all attributes")
plt.show()
```

Decision tree trained on all attributes



```
In [31]: print(tree_clf.predict([[.5, 1.5,5,7.5]]))
print(tree_clf.predict([[.5, 1.5,2.5,1.2]]))
print(tree_clf.predict([[5, 1.5,2,3]]))
```

```
In [31]: print(tree_clf.predict([[.5, 1.5,5,7.5]]))
print(tree_clf.predict([[.5, 1.5,2.5,1.2]]))
print(tree_clf.predict([[5, 1.5,2,3]]))
```

```
[2]
[1]
[0]
```

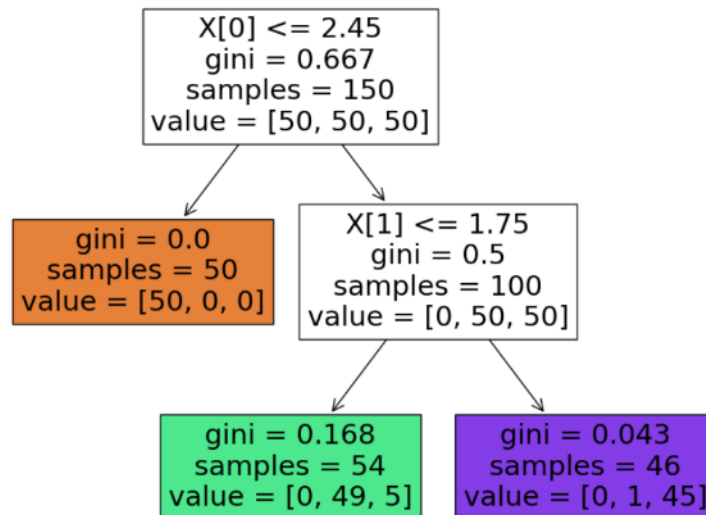


Decision Tree for the Iris dataset: using two attributes

```
In [15]: >> iris = load_iris(as_frame=True)
columns_to_use = ["petal length (cm)", "petal width (cm)"]
X_iris = iris.data[columns_to_use].values
y_iris = iris.target
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf = tree_clf.fit(X_iris, y_iris)
```

```
In [18]: >> plt.figure(figsize=(10,8))
plot_tree(tree_clf, filled=True)
plt.title("Decision tree trained on two attributes")
plt.show()
```

Decision tree trained on two attributes



```
In [19]: >> tree_clf.predict([[3, 2.5]])
```

```
Out[19]: array([2])
```

```
In [16]: >> tree_clf.predict([[5, 1.5]])
```

```
Out[16]: array([1])
```

```
In [17]: >> tree_clf.predict([[.5, 1.5]])
```

```
Out[17]: array([0])
```

```
In [20]: >> r = export_text(tree_clf, feature_names=columns_to_use)
print(r)
```

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.75
|       |--- class: 1
|   |--- petal width (cm) > 1.75
|       |--- class: 2
```



App Recommendation System using a DT

What to recommend for?

Gender	Age	App
Female	15	
Female	25	
Male	32	
Female	35	
Male	12	
Male	14	



Female, 16 years old



Female, 30 years old



Male, 35 years old



Which feature is more important?

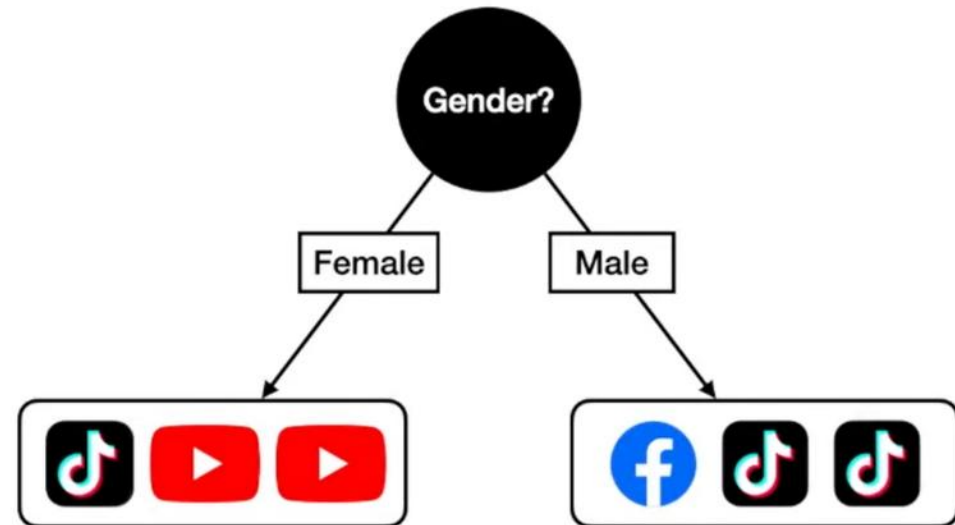
- To build a good decision tree, we must choose the **best feature to split on** at the root
- We compare two simple **decision stumps**:
 -  Gender-based stump (splits the data by gender)
 -  Age-based stump (splits the data by age)
- We evaluate each stump based on its **classification performance** (accuracy / error rate)
 - This tells us which feature **separates the classes better**

Goal

- Identify the feature that provides the **most informative first split** → the **root node** of the tree

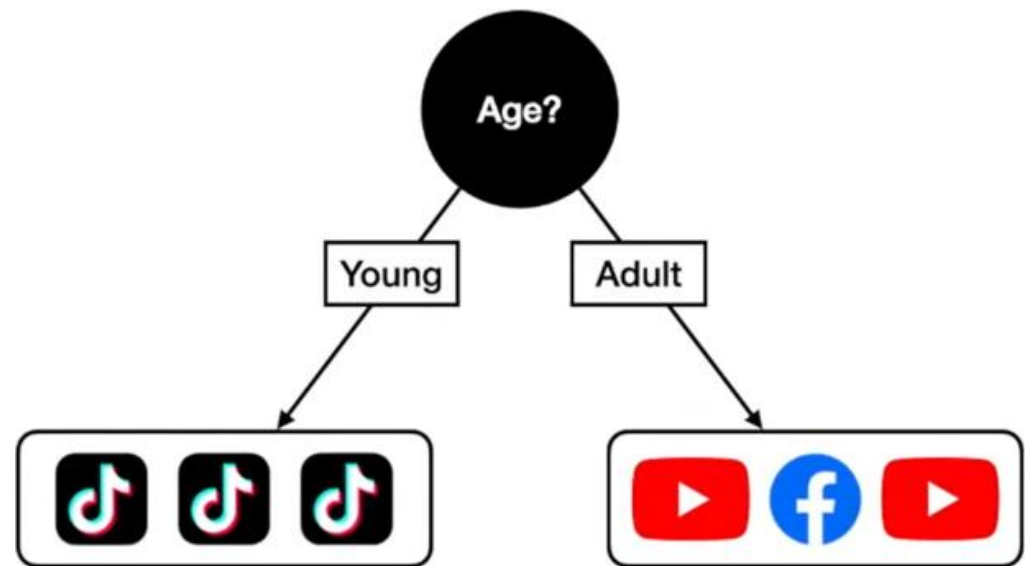
Splitting by Gender => Gender decision stump

Gender	App
Female	
Female	
Female	
Male	
Male	
Male	



Splitting by Age => Age decision stump

Age	App
Young	
Young	
Young	
Adult	
Adult	
Adult	

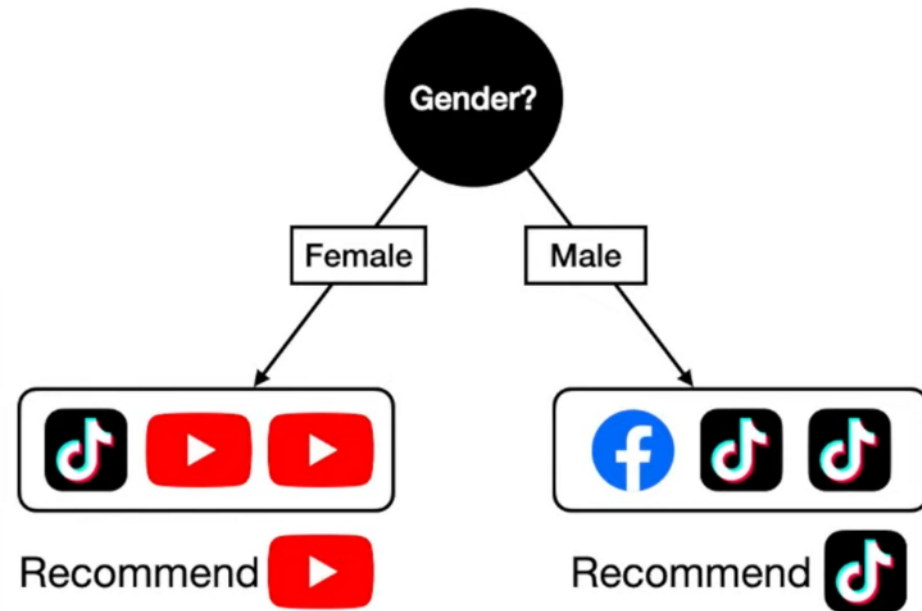


Young = Age $\leq$ 18

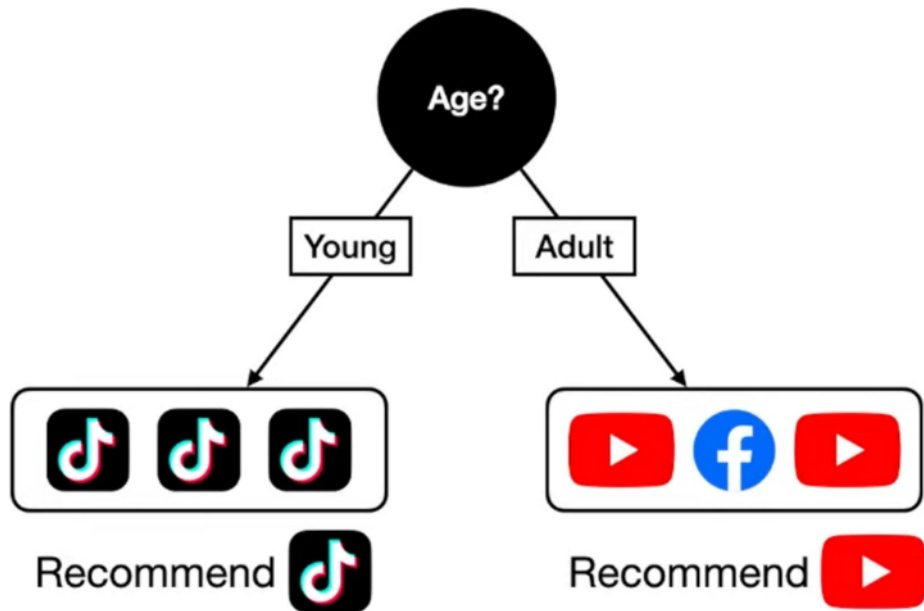
Adult : Age $>$ 18

Which one is better? 🤔

Gender decision stump



Age decision stump

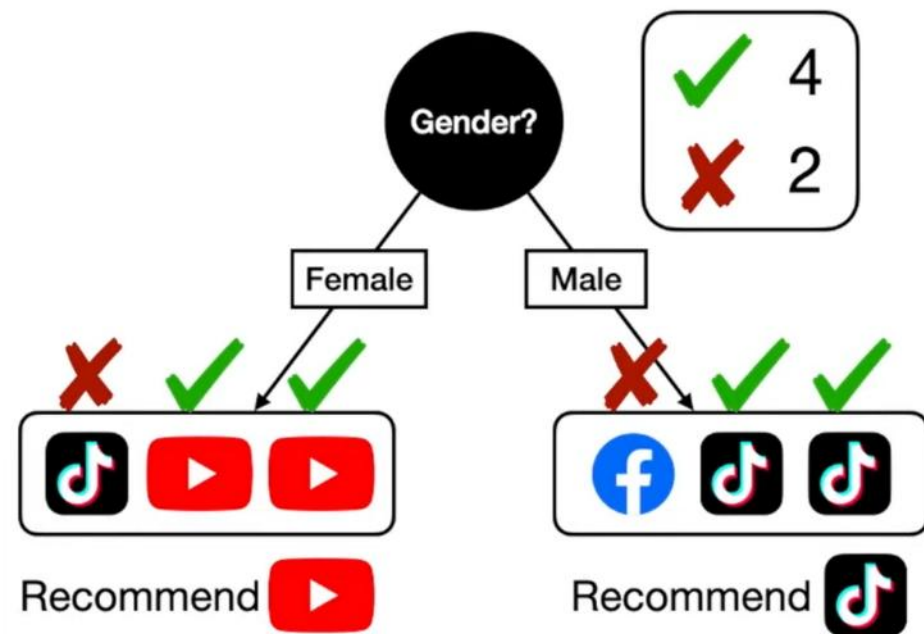


- Each feature creates a different **split** of the data, producing two smaller subsets
- We compare how well each split separates the classes using an **Attribute Selection Measure (ASM)** such **Classification Error**

Which one is better?

Accuracy: 66.7% & Classification error: 33.3%

Gender decision stump



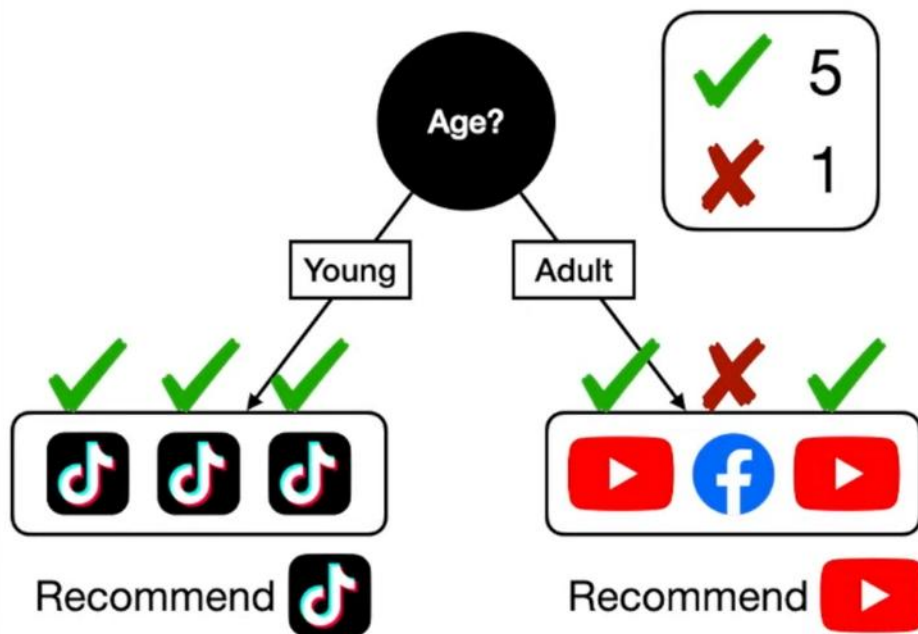
Accuracy: 83.3%

&



Classification error: 16.7%

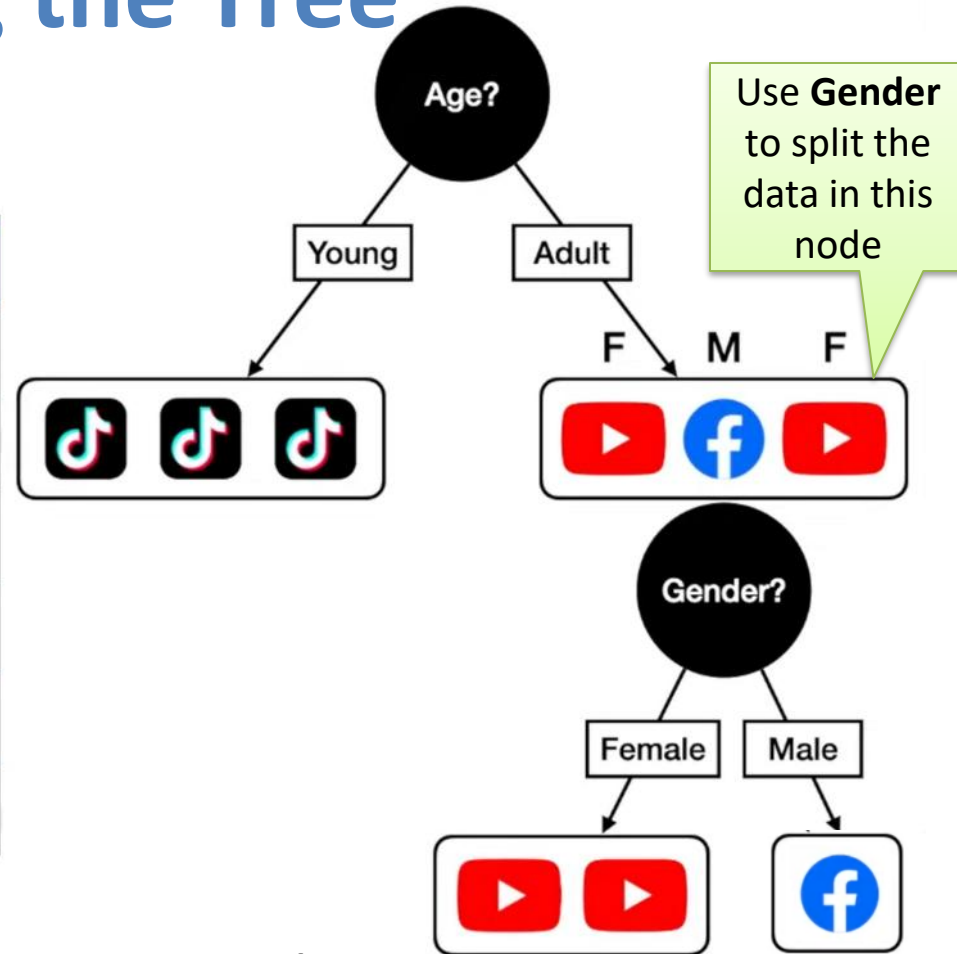
Age decision stump



- Based on **accuracy**, the **Age** feature provides the **best split** => It is **determinant** in differentiating the classes and deserve to be the **root node of the tree** 🏆 (it has the highest accuracy and the **lowest classification error rate**)
 - It is more effective at determining which app to recommend 🎯

Building the Tree

Gender	Age	App
Female	Young	
Female	Adult	
Male	Adult	
Female	Adult	
Male	Young	
Male	Young	



- Let's test it for recommending an app to:
 - Female, 16 years old 
 - Female, 30 years old 
 - Male, 35 years old 

Building the Tree – Key Decisions

1. **Choose the Best Feature to Split On (i.e., asking the best question):**
 - At each node, **select the feature that best separates the data** into distinct classes by reducing **impurity** the most
 - Use **Attribute Selection Measure (ASM)** like **Classification error**, **Gini impurity**, **Entropy**
2. **Determine the Split Point** ✂
 - Make the selected feature a decision node then find the **optimal Split Point**, find the **splitting condition/threshold** that minimizes the impurity using the same ASM
3. **Recursive Split the Data** ↺ **until a stop condition:**
 - Recursively split the dataset into subsets based on the selected best **feature** and its **split point**. Continues **until a stopping condition is met**, such as:
 - ↓ Maximum tree depth reached
 - 📊 No significant impurity reduction
 - 📉 Too few instances to split

Decision Tree Construction Algorithms

Several algorithms exist for tree construction (aka **Tree induction**), differing mainly in the Attribute Selection Measure (ASM) they use:

- **CART Algorithm** (Classification and Regression Trees)
 - Produces binary decision tree
 - Uses Gini Impurity Index as ASM
- **ID3** (Iterative Dichotomiser 3)
 - Uses **Entropy** and **Information Gain** as ASM
- **CHAID Algorithm** (Chi-squared Automatic Interaction Detection)
 - Use Chi-square test (χ^2) as ASM

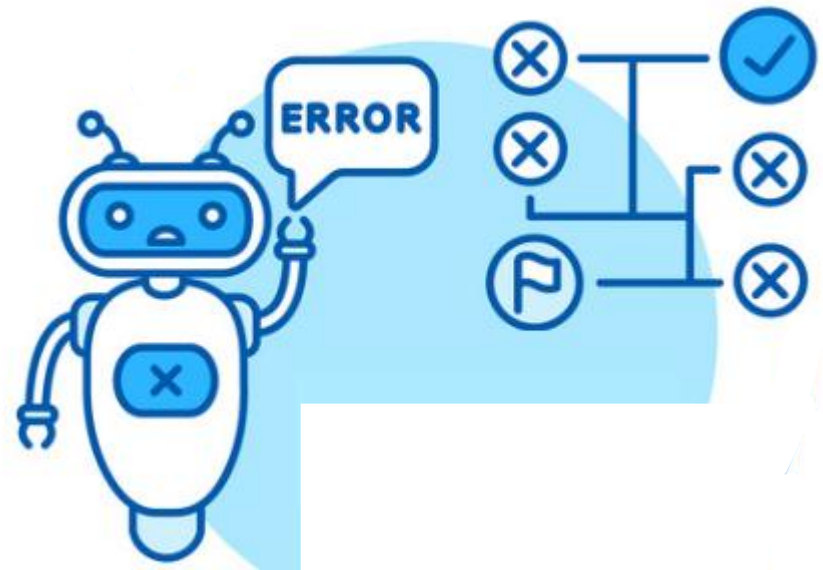
 Note: Empirical studies show that different ASMs often yield similar predictive accuracy 

Attribute Selection Measure (ASM)

Classification Error Rate

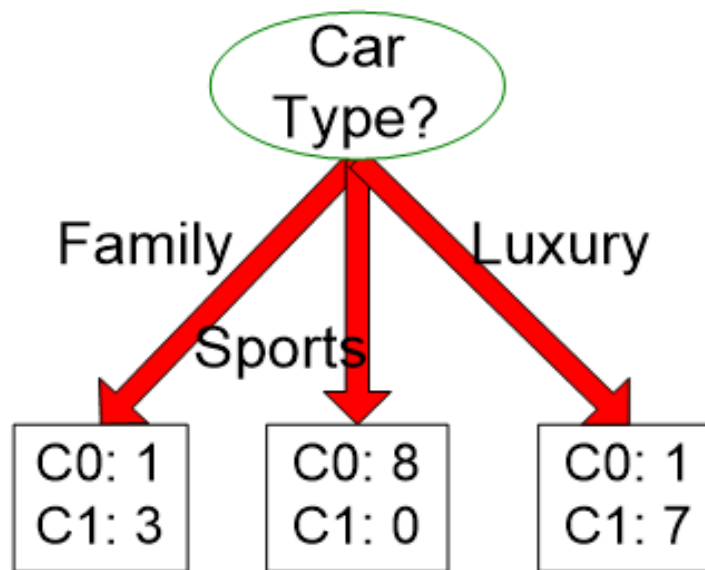
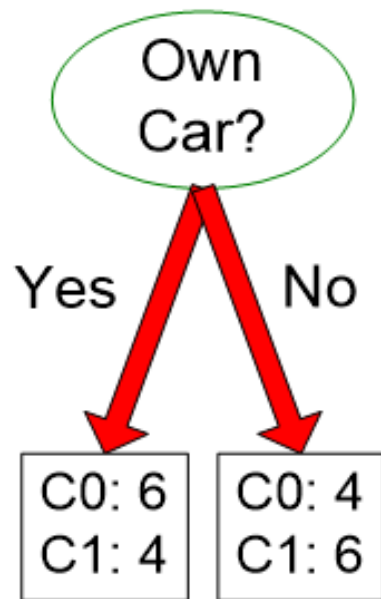
$$Error(t) = 1 - \max_i P(i|t)$$

Where $P(i|t)$ is the probability of class i at node t



How to choose the best Feature to Split On?

Before Splitting: 10 records of Class 0 (**Not Defaulted Borrower**)
10 records of Class 1 (**Defaulted Borrower**)



Which best feature to split on?

=> We need an **Attribute Selection Measure (ASM)**
for choosing the best feature to split that **maximizes the separation of classes**
(i.e., **minimizes impurity within each subset**)

How to choose the best Feature to Split On?

- Nodes with **homogeneous** class distribution (having **low impurity**) are preferred

C0: 5 C1: 5

**Non-homogeneous,
High impurity**

C0: 9 C1: 1

**Homogeneous,
Low impurity**

- Need an **Attribute Selection Measure (ASM)** to measure **the node impurity** and compare features to choose the best Feature to Split On
 - Classification Error Rate
 - **Gini Impurity Index** (or Gini index), and **Entropy** are measures of node impurity
- Then choose the feature that **minimizes impurity** within each subset (i.e., maximizes the **separation** of classes)

Classification Error Rate

- The Classification Error Rate (aka **Misclassification Rate**) measures how often a node incorrectly predicts the class

$$\text{Classification Error Rate} = \frac{\text{Number of Misclassified Instances}}{\text{Total Number of Instances}}$$

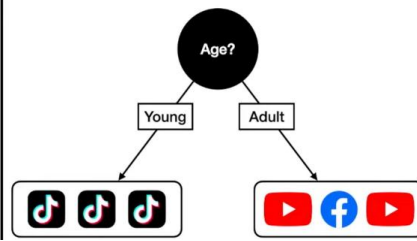
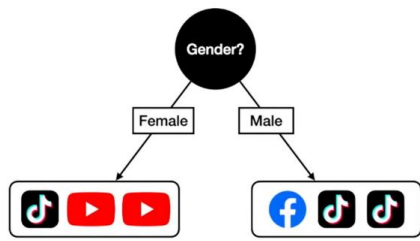
$$\text{Error}(t) = 1 - \max_i P(i|t)$$

Where $P(i|t)$ is the probability of class i at node t

- **Error(t)** measures the classification error made by a node
 - Proportion of the instances in the node that do not belong to the majority class
 - Minimum **0** → Pure node (all instances belong to one class)
 -  High → Mixed node with uncertain predictions
 - Maximum $(1 - \frac{1}{k})$ when all k classes are equally likely

The DT algorithms selects the feature that minimizes the Classification Error Rate

Examples for Computing Classification Error Rate



$$Error(t) = 1 - \max_i P(i | t)$$

Weighted Average

$$\bar{x}_{weighted} = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

Female (Left)	T	1
	Y	2

$$P(T) = \frac{1}{3}$$

$$P(Y) = \frac{2}{3}$$

$$Error(L) = 1 - \max\left(\frac{1}{3}, \frac{2}{3}\right) = 1 - \frac{2}{3} = \frac{1}{3}$$

Male (Right)	F	1
	T	2

$$P(F) = \frac{1}{3}$$

$$P(T) = \frac{2}{3}$$

$$Error(R) = 1 - \max\left(\frac{1}{3}, \frac{2}{3}\right) = 1 - \frac{2}{3} = \frac{1}{3}$$

$$Error(Gender) = \frac{1}{3} = 33.3\%$$

Young (Left)	T	3

$$P(T) = \frac{3}{3}$$

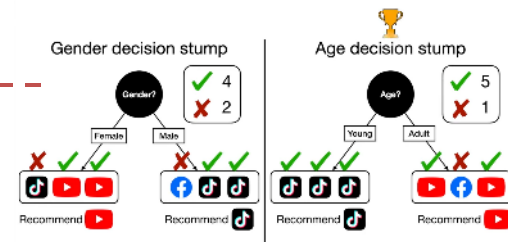
$$Error(L) = 1 - \max(1) = 1 - 1 = 0$$

Adult (Right)	Y	2
	F	1

$$P(Y) = \frac{2}{3}$$

$$P(F) = \frac{1}{3}$$

$$Error(R) = 1 - \max\left(\frac{2}{3}, \frac{1}{3}\right) = 1 - \frac{2}{3} = \frac{1}{3}$$

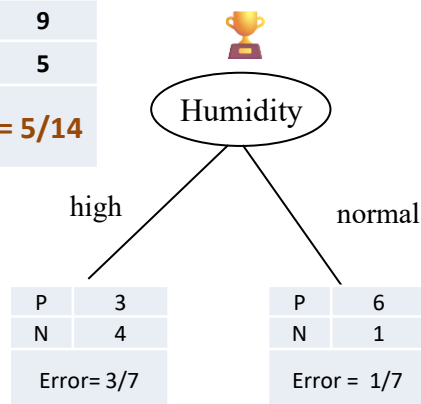


$$Error(Age) = \frac{1}{6} = 16.7\%$$



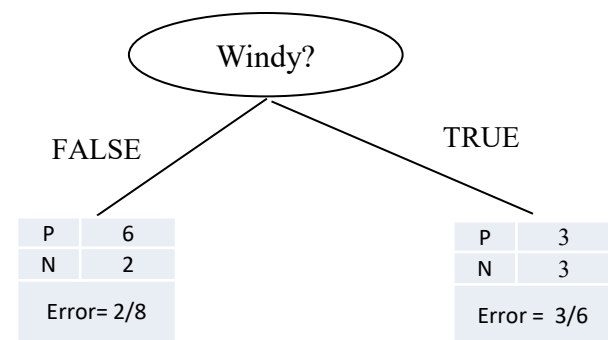
Outlook	Temperature	Humidity	Windy	Class
sunny	Hot	high	FALSE	N
sunny	Hot	High	TRUE	N
Overcast	Hot	high	FALSE	P
Rain	Mild	high	FALSE	P
Rain	Cool	Normal	FALSE	P
Rain	Cool	Normal	TRUE	N
Overcast	Cool	Normal	TRUE	P
Sunny	Mild	High	FALSE	N
Sunny	Cool	normal	FALSE	P
Rain	Mild	Normal	FALSE	P
Sunny	Mild	Normal	TRUE	P
Overcast	Mild	High	TRUE	P
Overcast	Hot	Normal	FALSE	P
Rain	mild	high	TRUE	N

	Parent
P	9
N	5
Error= 5/14	



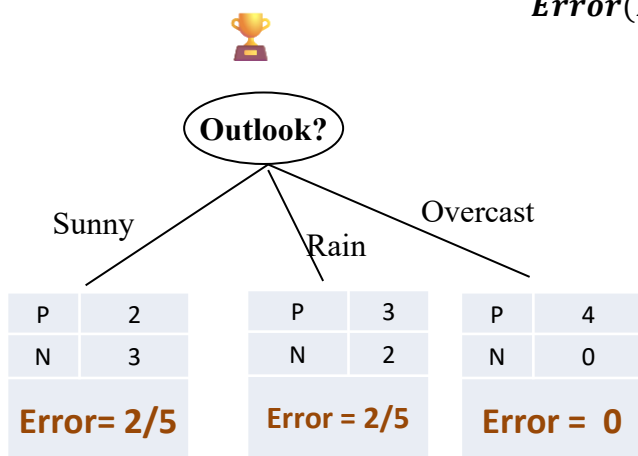
Weighted Average Error for the Humidity Split

$$Error(Humidity) = \frac{7}{14} \times \frac{3}{7} + \frac{7}{14} \times \frac{1}{7} = \frac{3}{14} + \frac{1}{14} = \frac{4}{14}$$



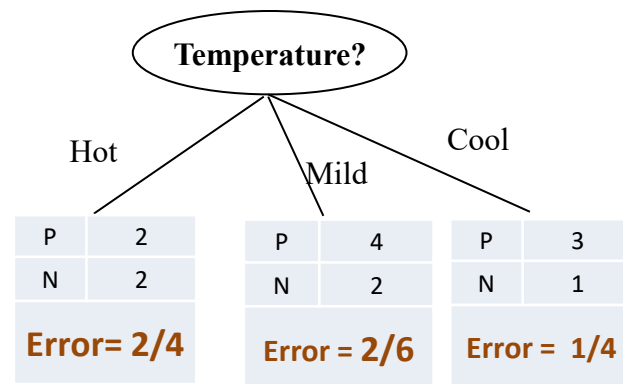
Weighted Average Error for the Windy Split

$$Error(Windy) = \frac{8}{14} \times \frac{2}{8} + \frac{6}{14} \times \frac{3}{6} = \frac{5}{14}$$



Weighted Average Error for the Outlook Split

$$Error(Outlook) = \frac{5}{14} \times \frac{2}{5} + \frac{5}{14} \times \frac{2}{5} + \frac{4}{14} \times 0 = \frac{2}{14} + \frac{2}{14} = \frac{4}{14}$$

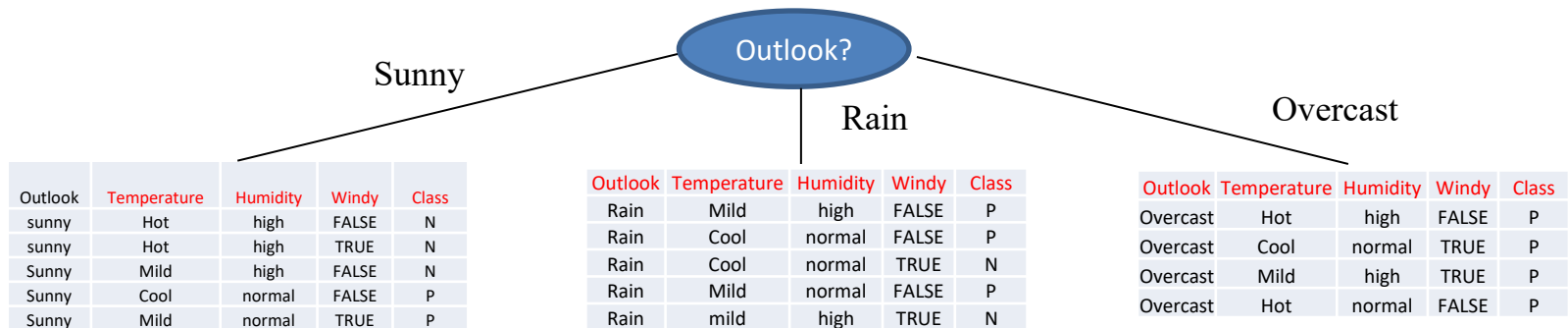


Weighted Average Error for the Temperature Split

$$Error(Temperature) = \frac{4}{14} \times \frac{2}{4} + \frac{6}{14} \times \frac{2}{6} + \frac{4}{14} \times \frac{1}{4} = \frac{2}{14} + \frac{2}{14} + \frac{1}{14} = \frac{5}{14}$$

The best two splits are Outlook and Humidity
We can use Outlook or Humidity since both have the same error

Outlook	Temperature	Humidity	Windy	Class
sunny	Hot	high	FALSE	N
sunny	Hot	high	TRUE	N
Overcast	Hot	high	FALSE	P
Rain	Mild	high	FALSE	P
Rain	Cool	normal	FALSE	P
Rain	Cool	normal	TRUE	N
Overcast	Cool	normal	TRUE	P
Sunny	Mild	high	FALSE	N
Sunny	Cool	normal	FALSE	P
Rain	Mild	normal	FALSE	P
Sunny	Mild	normal	TRUE	P
Overcast	Mild	high	TRUE	P
Overcast	Hot	normal	FALSE	P
Rain	mild	high	TRUE	N



Next: Repeat selecting the best feature to split on

Attribute Selection Measure (ASM)

Gini Impurity Index

$$Gini(t) = 1 - \sum_{i=1}^k p_i(t)^2$$

Where $p_i(t)$ is the frequency of class i at node t , and k is the number of classes at node t



Low Gini
impurity index



High Gini
impurity index

Gini Index

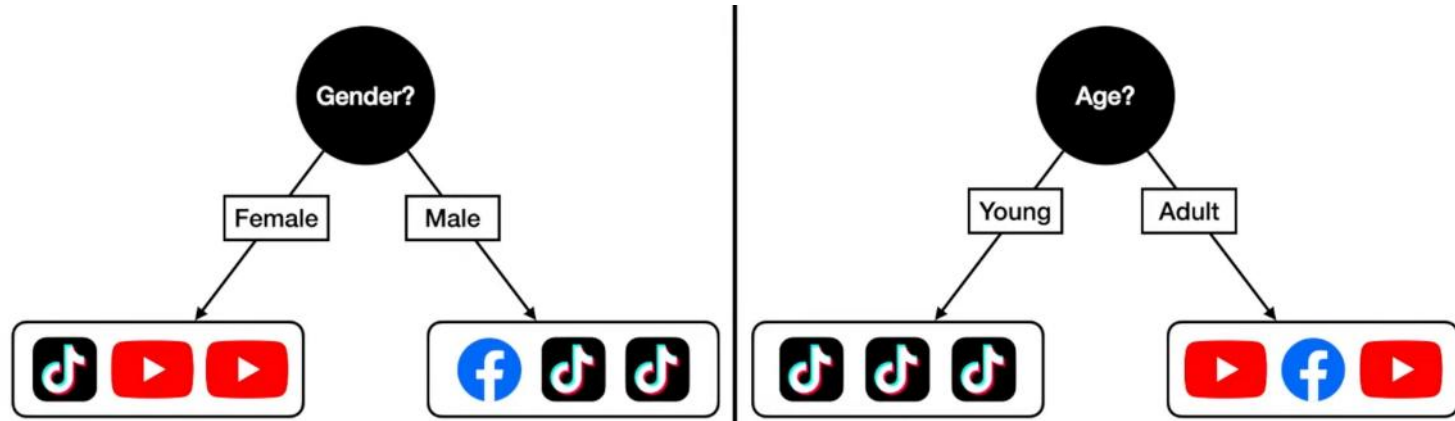
- The Gini Index measures how **impure** (mixed) a node is
- It ranges from:
 - 0** → Pure node (all instances from one class)
 - $1 - 1/k$** → Maximum impurity when classes are evenly distributed ($k = \text{number of classes}$)
- The formula to calculate the Gini Index for a node **t** with **k** classes is:

$$Gini(t) = 1 - \sum_{i=1}^k p_i(t)^2$$

Where $p_i(t)$ = proportion of class i at node t ,
and k = number of classes at node t

- DT algorithm selects the **feature** and **split point** that produces child nodes with the **lowest weighted Gini impurity**
→ purer splits → better classification

Which one is better? => Compute Gini Index



Classifier 1 (by Gender): Avg Gini = $((3 \times 0.44) + (3 \times 0.44)) / 6 = 0.44$

- Left leaf (Female): {T, Y, Y}

$$\text{Gini} = 1 - (\mathbf{P(T)^2} + \mathbf{P(Y)^2}) = 1 - \left(\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2 \right) = \mathbf{0.44}$$

- Right leaf (Male): {F, T, T}

$$\text{Gini} = 1 - (\mathbf{P(F)^2} + \mathbf{P(T)^2}) = 1 - \left(\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2 \right) = \mathbf{0.44}$$

Measures the
impurity of
the split

Classifier 2 (by age): Avg Gini = $((3 \times 0) + (3 \times 0.44)) / 6 = 0.22$



- Left leaf (young): {T, T, T}. **Gini** = $1 - \mathbf{P(T)^2} = 1 - \left(\frac{3}{3}\right)^2 = \mathbf{0}$

- Right leaf (adult): {Y, F, Y}. **Gini** = $1 - (\mathbf{P(Y)^2} + \mathbf{P(F)^2}) = 1 - \left(\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2 \right) = \mathbf{0.44}$

Compute Gini Index – Example 2

$$Gini(t) = 1 - \sum_{i=1}^K p(i|t)^2$$

Where $p(i|t)$ is the probability of class i at node t

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Gini = 1 - P(C1)^2 - P(C2)^2 = 1 - 0 - 1 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Gini = 1 - (1/6)^2 - (5/6)^2 = 0.278$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Gini = 1 - (2/6)^2 - (4/6)^2 = 0.444$$

Splitting Based on Gini

- When a parent node p is split into k child nodes, the quality of the split is measured using the **weighted Gini impurity**:

$$Gini_{split} = \sum_{i=1}^k \frac{n_i}{n} Gini(i)$$

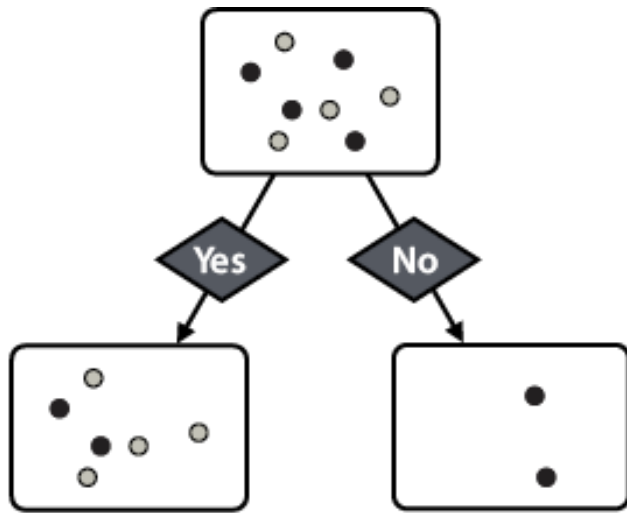
Where:

- n_i = number of instances in child node i
- n = total number of instances in the parent node
- $Gini(i)$ = Gini impurity of child node i

 Goal: Choose the split that results in the **lowest weighted Gini impurity**, producing the purest child nodes

Gini Index of a Node => take weighted average

- A split of a Node of size 8 into 2 partitions of sizes 6 and 2
- We calculate the Gini index of the Node as the **weighted average** of the Gini of partitions:
 - We weight the index of the left partition by 6/8 and that of the right partition by 2/8



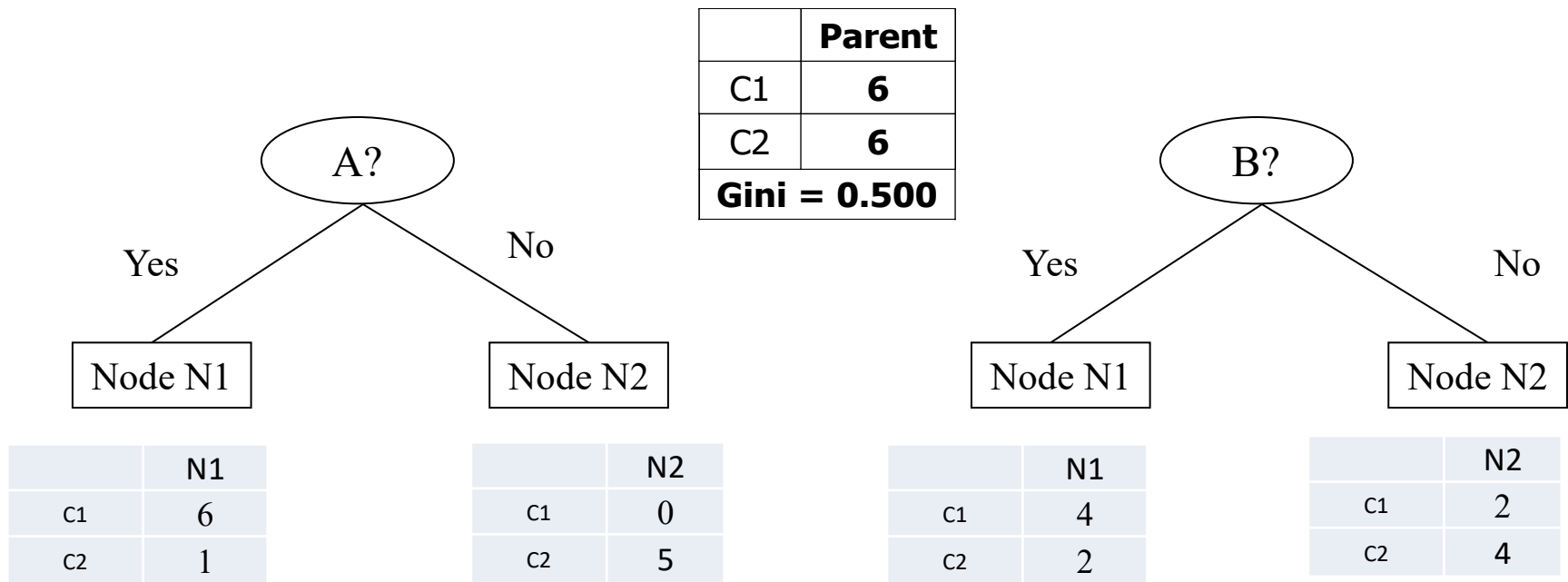
Gini = 0.444

Gini = 0

$$\text{Weighted average Gini} = 0.444 \cdot \frac{6}{8} + 0 \cdot \frac{2}{8} = 0.333$$

Computing Gini Index of a Split

- Split into two partitions
- Compute the Gini Index per split
- Choose the split with the lowest Gini Index



$$\text{Gini}(N1) = 1 - (6/7)^2 - (1/7)^2 = 0.245$$

$$\text{Gini}(N2) = 1 - (0/5)^2 - (5/5)^2 = 0$$

$$\text{Gini}(A) = 7/12 * 0.245 + 5/12 * 0 = \mathbf{0.1429}$$

$$\text{Gini}(N1) = 1 - (4/6)^2 - (2/6)^2 = 0.2222$$

$$\text{Gini}(N2) = 1 - (2/6)^2 - (4/6)^2 = 0.2222$$

$$\text{Gini}(B) = 6/12 * 0.2222 + 6/12 * 0.2222 = \mathbf{0.2222}$$

Choose split A as it has a Gini Index < split B

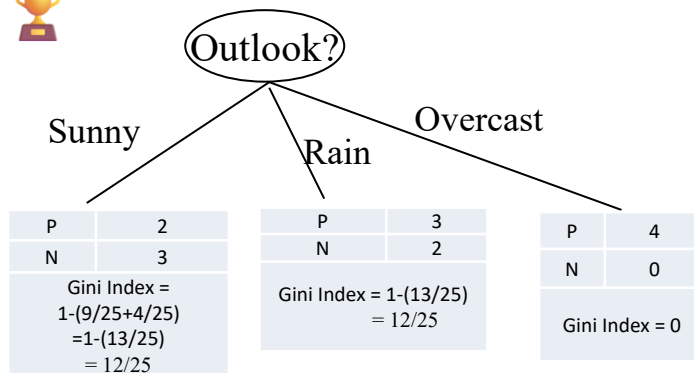
Computing Gini Index – Example (1/2)

$$Gini(t) = 1 - \sum_{i=1}^K p(i|t)^2$$

Where $p(i|t)$ is the probability of class i at node t

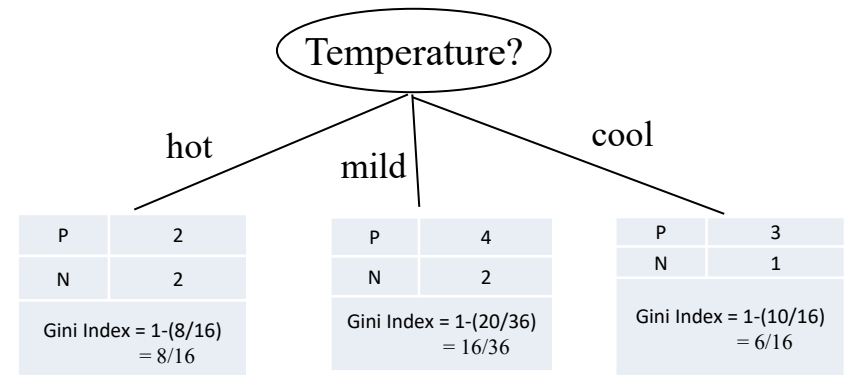
Outlook	Temperature	Humidity	Windy	Class
sunny	Hot	high	FALSE	N
sunny	Hot	high	TRUE	N
Overcast	Hot	high	FALSE	P
Rain	Mild	high	FALSE	P
Rain	Cool	normal	FALSE	P
Rain	Cool	normal	TRUE	N
Overcast	Cool	normal	TRUE	P
Sunny	Mild	high	FALSE	N
Sunny	Cool	normal	FALSE	P
Rain	Mild	normal	FALSE	P
Sunny	Mild	normal	TRUE	P
Overcast	Mild	high	TRUE	P
Overcast	Hot	normal	FALSE	P
Rain	mild	high	TRUE	N

	Parent
P	9
N	5
Gini Index = $1 - (25/196 + 81/196)$ = $90/196 = 0.4592$	



Weighted Average Gini Index for the Outlook Split

$$Gini(Outlook) = \frac{5}{14} \times \frac{12}{25} + \frac{5}{14} \times \frac{12}{25} + \frac{4}{14} \times 0 = \frac{6}{35} + \frac{6}{35} = \frac{12}{35} = 0.3429$$



Weighted Average Gini Index for the Temperature Split

$$Gini(Temperature) = \frac{4}{14} \times \frac{8}{16} + \frac{6}{14} \times \frac{16}{36} + \frac{4}{14} \times \frac{6}{16}$$

$$Gini(Temperature) = \frac{2}{14} + \frac{4}{21} + \frac{3}{28} = \frac{4}{21} + \frac{7}{28} = 0.4405$$

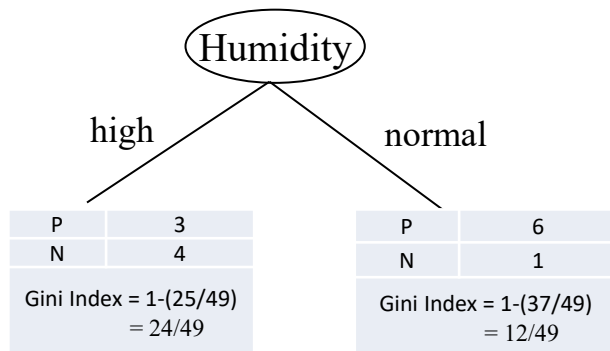
Computing Gini Index – Example (2/2)

$$Gini(t) = 1 - \sum_{i=1}^k p_i(t)^2$$

Where $p_i(t)$ is the frequency of class i at node t , and k is the number of classes at node t

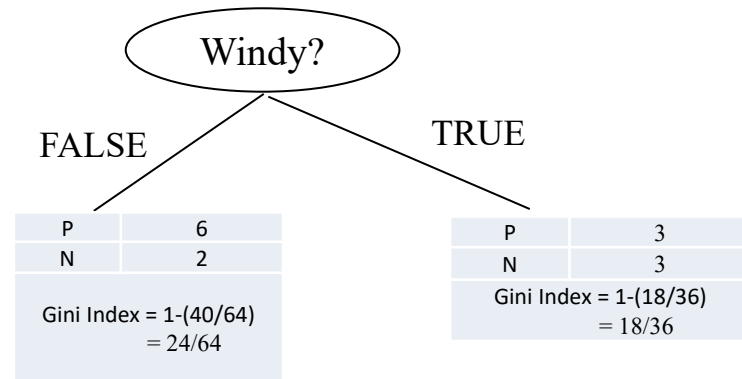
Outlook	Temperature	Humidity	Windy	Class
sunny	Hot	high	FALSE	N
sunny	Hot	high	TRUE	N
Overcast	Hot	high	FALSE	P
Rain	Mild	high	FALSE	P
Rain	Cool	normal	FALSE	P
Rain	Cool	normal	TRUE	N
Overcast	Cool	normal	TRUE	P
Sunny	Mild	high	FALSE	N
Sunny	Cool	normal	FALSE	P
Rain	Mild	normal	FALSE	P
Sunny	Mild	normal	TRUE	P
Overcast	Mild	high	TRUE	P
Overcast	Hot	normal	FALSE	P
Rain	mild	high	TRUE	N

	Parent
P	9
N	5
Gini Index = $1 - (25/196 + 81/196)$ = $90/196 = 0.4592$	



Weighted Average Gini Index for the Humidity Split

$$Gini(Outlook) = \frac{7}{14} \times \frac{24}{49} + \frac{7}{14} \times \frac{12}{49} = \frac{12}{49} + \frac{6}{49} = \frac{18}{49} = 0.3673$$



Weighted Average Gini Index for the Windy Split

$$Gini(Temperature) = \frac{8}{14} \times \frac{24}{64} + \frac{6}{14} \times \frac{18}{36} = \frac{3}{14} + \frac{3}{14} = \frac{6}{14} = 0.4286$$

The best split is Outlook (0.3429)

Decide optimal **Split Point** using Gini Index for Categorical Attributes

-  For each distinct value, count how many instances belong to each class
-  Use this count matrix to evaluate possible **split points** and choose the best one

Multi-way split



	CarType		
	Family	Sports	Luxury
C1	1	2	1
C2	4	1	1
Gini	0.393		

Two-way split
(find best partition of values)

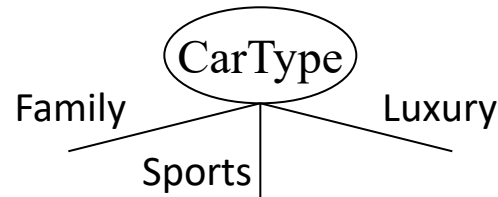
	CarType	
	{Sports, Luxury}	{Family}
C1	3	1
C2	2	4
Gini	0.400	

	CarType	
	{Sports}	{Family, Luxury}
C1	2	2
C2	1	5
Gini	0.419	

Splitting Based on Nominal Attributes

Nominal (categorical, unordered) attributes can be split in two main ways:

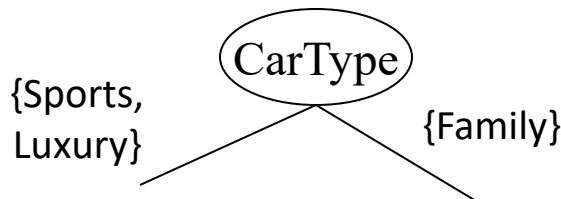
- **Multi-way Split** : Each distinct value becomes its own branch



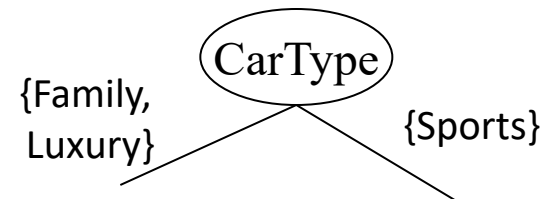
– This keeps the tree simple but may create many branches.

- **Binary Split** : Group the values into two subsets and split the data accordingly

We search for the best grouping that gives the lowest impurity (Gini/Entropy/Error)



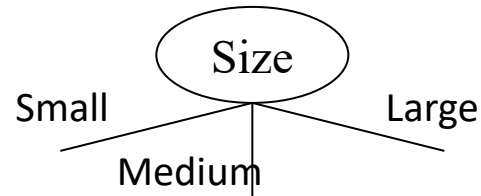
OR



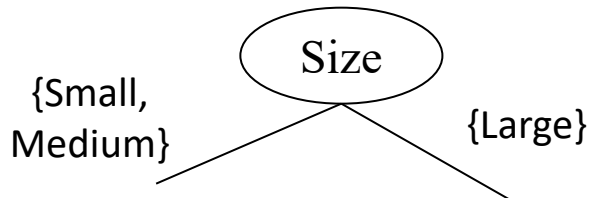
- Binary splits can improve performance but require searching over multiple possible groupings

Splitting Based on Ordinal Attributes

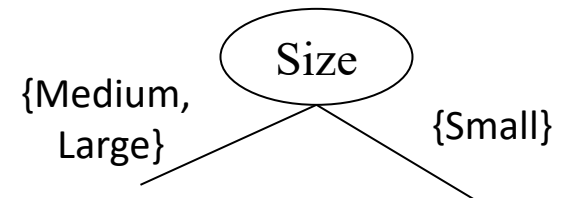
- **Multi-way Split** : Use as many partitions as distinct values



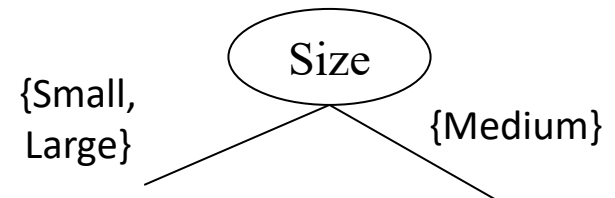
- **Binary Split** : Divides values into two subsets
Need to find optimal partitioning



OR



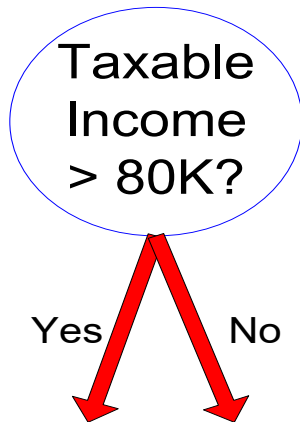
- What about this split?
No! the grouping should not violate the order property of the attribute values



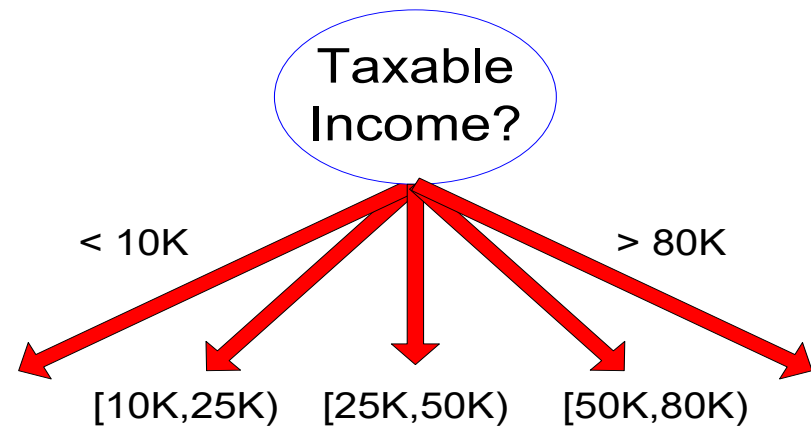
Splitting Based on Continuous Attributes

- Continuous features can be handled in several ways:
 - **Discretization** to form an ordinal categorical attribute
 - Ranges can be found using Equal-interval binning, Equal-frequency binning, or Clustering-based binning
- **Binary Split** 
Sort the values of attribute **A**, then consider **midpoints between consecutive values** as candidate split points:
 - Candidate threshold = $(a_i + a_{i+1})/2$
 - Evaluate each threshold and choose the one that **minimizes impurity** (Gini/Entropy/Error)

Splitting Based on Continuous Attributes



(i) Binary split

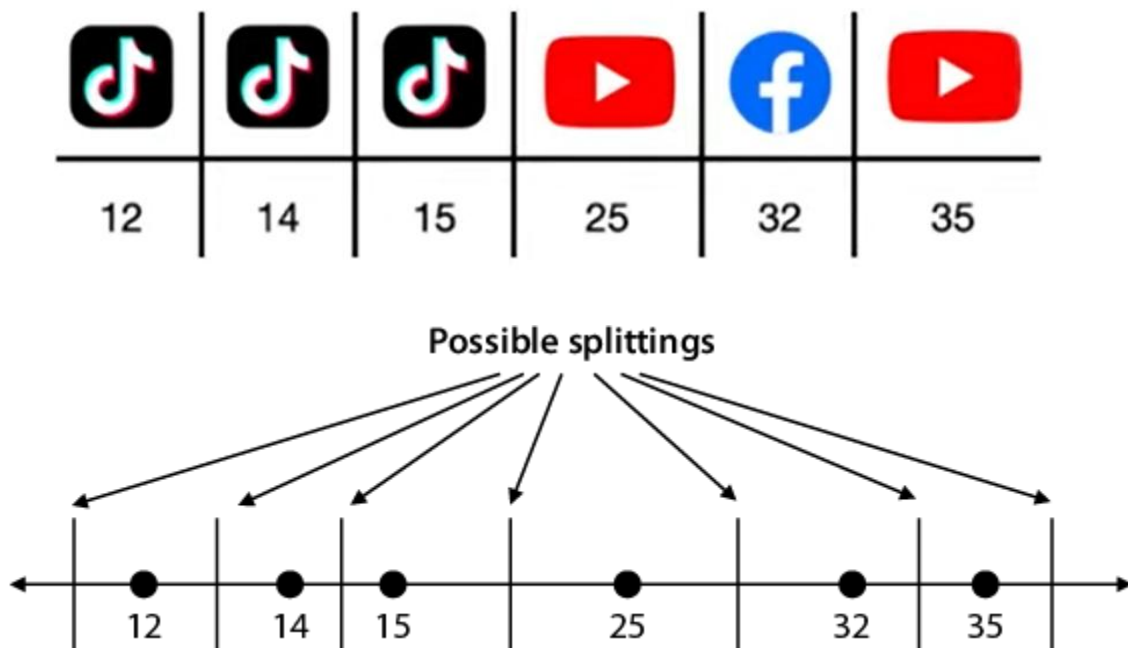


(ii) Multi-way split

Decide optimal **Split Point** using Gini Index

Continuous Attributes

Gender	Age	App
Female	15	
Female	25	
Male	32	
Female	35	
Male	12	
Male	14	



- Sort the entries by **age**
- We pick the **midpoints** between consecutive ages to be the age for **splitting**
 - For the endpoints, we can pick any random value that is out of the interval
- Then calculate the Gini impurity index of each of the splits
- Choose the **Split Point** that has the lowest Gini index

Decide optimal **Split Point** using Gini Index - Example



	Age													
	12		14		15		25		32		35			
App	7		13		14.5		20		28.5		33.5		100	
	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
T	0	3	1	2	2	1	3	0	3	0	3	0	3	0
Y	0	2	0	2	0	2	0	2	1	1	2	1	2	0
F	0	1	0	1	0	1	0	1	2	0	1	0	1	0
Gini	0.611		0.533		0.417		0.22		0.416		0.467		0.611	



The best **Split Point** is age ≤ 20

Attribute Selection Measure (ASM)

Entropy

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5



Entropy(PlayGolf) = Entropy(5,9)
= Entropy(0.36, 0.64)
= - (0.36 log₂ 0.36) - (0.64 log₂ 0.64)
= 0.94

Entropy

- Entropy is a **measure of impurity** or randomness in a dataset. It quantifies the **amount of uncertainty** associated with the distribution of class labels in the dataset

- The formula to calculate entropy for a set S with K classes is:

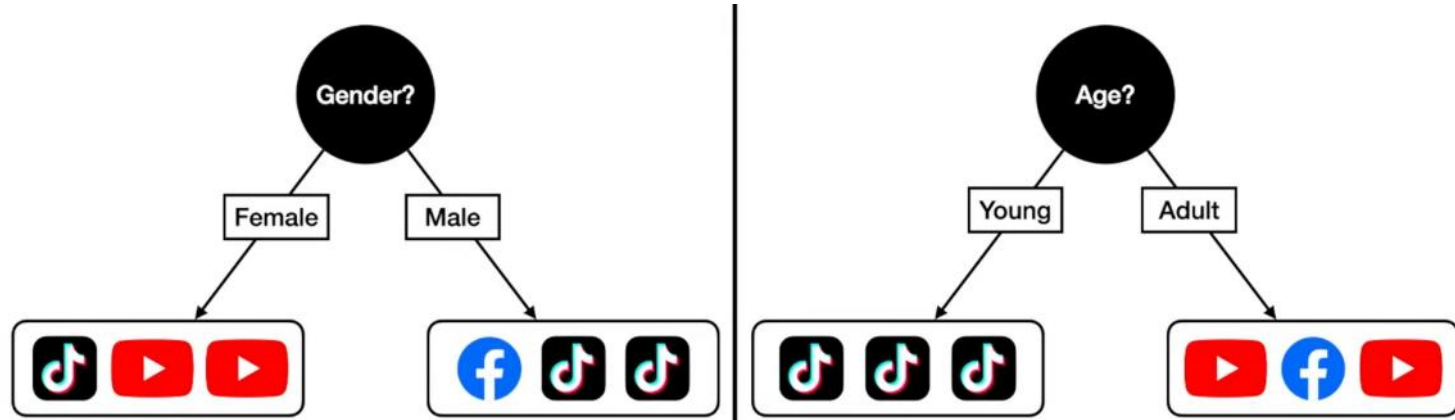
$$\text{Entropy}(S) = - \sum_{i=1}^K p_i \log_2(p_i)$$

Where p_i is the probability of class i in set S .

Fraction of instances
of a given class....

- Entropy ranges from 0 to $\log_2(K)$, where:
 - 0 indicates that the set S is pure (all instances belong to the same class)
 - $\log_2(K)$** indicates maximum entropy (the instances are evenly distributed across all classes)
- DT algorithm selects the feature and split point that **result in subsets with lower entropy**

Which one is better? => Compute Entropy



Classifier 1 (by Gender): Avg Entropy = $((3 \times 0.918) + (3 \times 0.918))/6 = 0.918$

- Left leaf (Female): {T, Y, Y}

$$\text{Entropy} = -\mathbf{P(T)} \log_2 \mathbf{P(T)} - \mathbf{P(Y)} \log_2 \mathbf{P(Y)} = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = \mathbf{0.918}$$

- Right leaf (Male): {F, T, T}

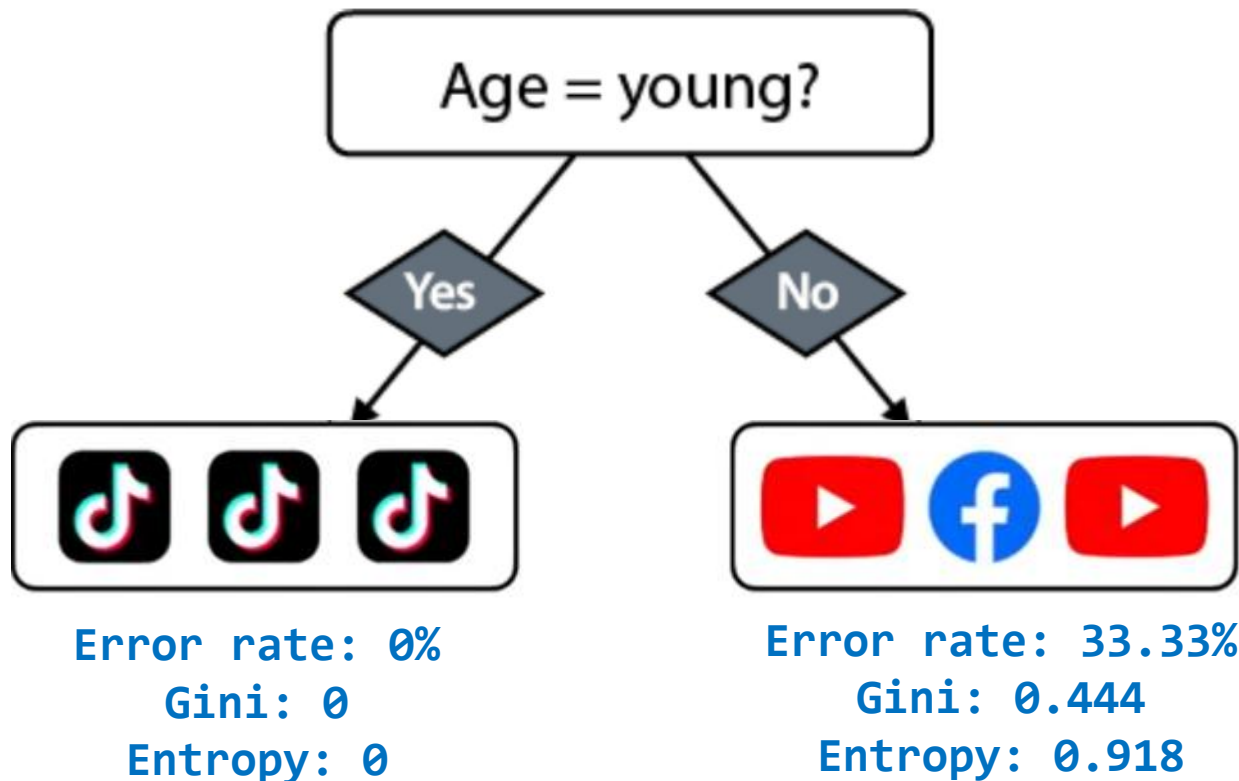
$$\text{Entropy} = -\mathbf{P(F)} \log_2 \mathbf{P(F)} - \mathbf{P(T)} \log_2 \mathbf{P(T)} = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.918$$

Classifier 2 (by age): Avg Entropy = $((3 \times 0) + (3 \times 0.918))/6 = 0.459$

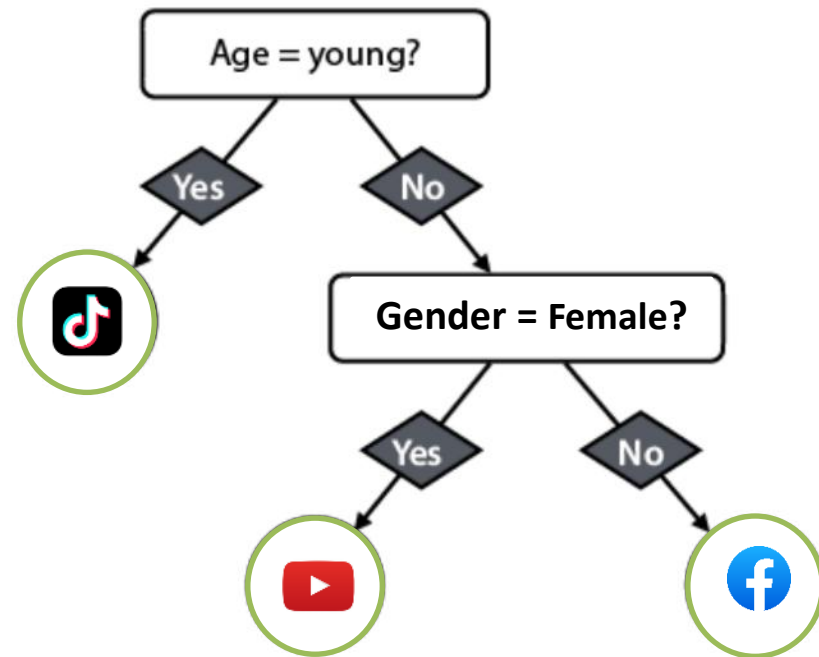
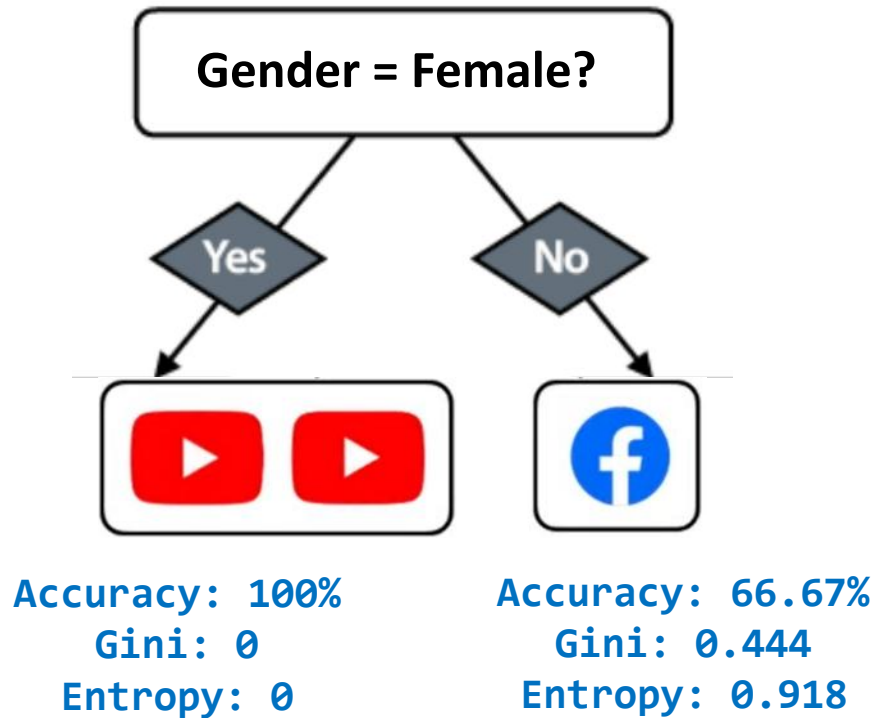


- Left leaf (young): {T, T, T}. **Entropy** = $-\mathbf{P(T)} \log_2 \mathbf{P(T)} = -\frac{3}{3} \log_2 \frac{3}{3} = \mathbf{0}$

- Right leaf (adult): {Y, F, Y}. **Entropy** = $-\mathbf{P(Y)} \log_2 \mathbf{P(Y)} - \mathbf{P(F)} \log_2 \mathbf{P(F)}$
 $= -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} = 0.918$



- When we split our dataset by age, we get two partitioned datasets
- The one on the left **is pure** (all the labels are the same), its error rate is 0%, and its Gini index and entropy are both 0
 - Thus, this node becomes **a leaf node**, and when we get to that leaf, we return the prediction **TikTok**
- The split on the right is **impure** and can still be divided using the Gender feature



- We can split the right leaf of the tree in the previous slide using Gender and we obtain two pure datasets. Each having an accuracy of 100% and a Gini index and entropy of 0
- After this split, we are done, because we can't improve our splits any further
- The resulting decision tree (shown on the right) has two nodes and three leaves. This tree predicts every point in the original dataset correctly

Entropy Calculation – Example 2 (1/5)

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	FALSE	N
sunny	hot	high	TRUE	N
overcast	hot	high	FALSE	P
rain	mild	high	FALSE	P
rain	cool	normal	FALSE	P
rain	cool	normal	TRUE	N
overcast	cool	normal	TRUE	P
sunny	mild	high	FALSE	N
sunny	cool	normal	FALSE	P
rain	mild	normal	FALSE	P
sunny	mild	normal	TRUE	P
overcast	mild	high	TRUE	P
overcast	hot	normal	FALSE	P
rain	mild	high	TRUE	N

$$\text{Entropy}(S) = - \sum_{i=1}^K p_i \log_2(p_i)$$

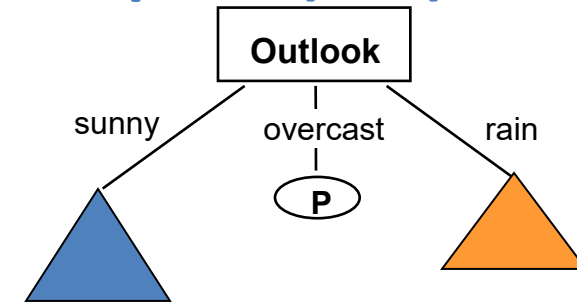
Where p_i is the probability of class i in set S .

$$E(S) = -\frac{9}{9+5} \cdot \log_2 \frac{9}{9+5} - \frac{5}{9+5} \cdot \log_2 \frac{5}{9+5} \approx 0.94$$

Entropy Calculation ID3 Algorithm – Example 2 (2/5)

$$E(S) = -\frac{9}{9+5} \cdot \log_2 \frac{9}{9+5} - \frac{5}{9+5} \cdot \log_2 \frac{5}{9+5} \approx 0.94$$

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	FALSE	N
sunny	hot	high	TRUE	N
overcast	hot	high	FALSE	P
rain	mild	high	FALSE	P
rain	cool	normal	FALSE	P
rain	cool	normal	TRUE	N
overcast	cool	normal	TRUE	P
sunny	mild	high	FALSE	N
sunny	cool	normal	FALSE	P
rain	mild	normal	FALSE	P
sunny	mild	normal	TRUE	P
overcast	mild	high	TRUE	P
overcast	hot	normal	FALSE	P
rain	mild	high	TRUE	N



sunny

overcast

rain

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	FALSE	N
sunny	hot	high	TRUE	N
sunny	mild	high	FALSE	N
sunny	cool	normal	FALSE	P
sunny	mild	normal	TRUE	P

Outlook	Temperature	Humidity	Windy	Class
overcast	hot	high	FALSE	P
overcast	cool	normal	TRUE	P
overcast	mild	high	TRUE	P
overcast	hot	normal	FALSE	P

Outlook	Temperature	Humidity	Windy	Class
rain	mild	high	FALSE	P
rain	cool	normal	FALSE	P
rain	cool	normal	TRUE	N
rain	mild	normal	FALSE	P
rain	mild	high	TRUE	N

$$E(\text{Sunny}) = -\frac{2}{5} \cdot \log_2 \frac{2}{5} - \frac{3}{5} \cdot \log_2 \frac{3}{5} \approx 0.971$$

$$E(\text{Overcast}) = -\frac{4}{4} \cdot \log_2 \frac{4}{4} = 0$$

$$E(\text{Rain}) = -\frac{3}{5} \cdot \log_2 \frac{3}{5} - \frac{3}{5} \cdot \log_2 \frac{3}{5} \approx 0.971$$

$$E(\text{Outlook}) = \frac{5}{14} \cdot E(\text{Sunny}) + \frac{4}{14} \cdot E(\text{Overcast}) + \frac{5}{14} \cdot E(\text{Rain}) = 0.694$$

$$\text{Gain}(\text{Outlook}) = E(S) - E(\text{Outlook}) = 0.94 - 0.694 = 0.246$$

Information gain **Gain(A)** is difference between the entropy before splitting the dataset **S** and the entropy after splitting based on the attribute **A**:

$$\text{Gain}(A) = \text{Entropy}(S) - \text{Entropy}(S, A)$$

Entropy Calculation ID3 Algorithm – Example 2 (3/5)

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	FALSE	N
sunny	hot	high	TRUE	N
overcast	hot	high	FALSE	P
rain	mild	high	FALSE	P
rain	cool	normal	FALSE	P
rain	cool	normal	TRUE	N
overcast	cool	normal	TRUE	P
sunny	mild	high	FALSE	N
sunny	cool	normal	FALSE	P
rain	mild	normal	FALSE	P
sunny	mild	normal	TRUE	P
overcast	mild	high	TRUE	P
overcast	hot	normal	FALSE	P
rain	mild	high	TRUE	N



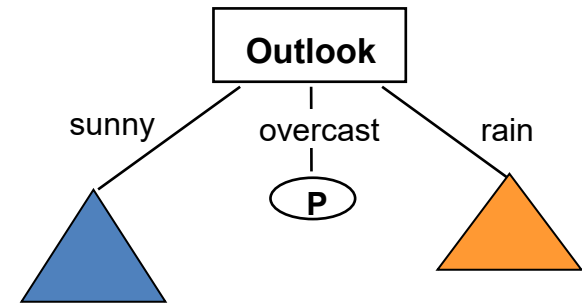
Gain(Outlook) = 0.246

Gain(Temperature) = 0.029

Gain(Humidity) = 0.151

Gain(Windy) = 0.048

∴ Outlook is chosen as the root



Choose the attribute with **the highest information gain** as the splitting attribute at the node i.e., the attribute that **provides the most reduction in uncertainty or impurity** in the dataset when used for splitting

Entropy Calculation ID3 Algorithm – Example 2 (4/5)

Training Set(outlook=Sunny)

Temperature	Humidity	Windy	Class
hot	high	FALSE	N
hot	high	TRUE	N
mild	high	FALSE	N
cool	normal	FALSE	P
mild	normal	TRUE	P

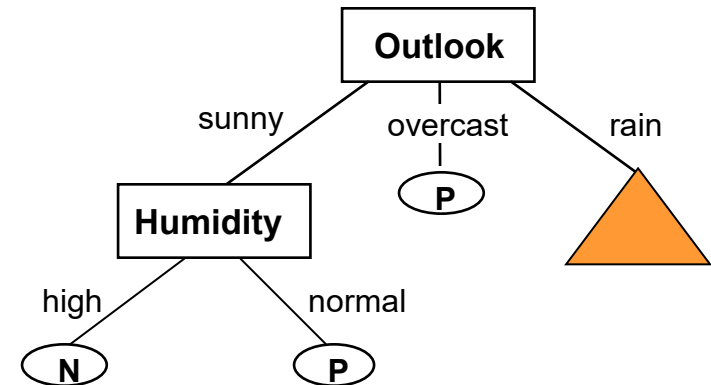


 **Gain(Temperature) = 0.571**

Gain(Humidity) = 0.971

Gain(Windy) = 0.020

∴ Humidity is chosen as the root



Entropy Calculation ID3 Algorithm – Example 2 (5/5)

Training Set(outlook=Rain)

Temperature	Humidity	Windy	Class
mild	high	FALSE	P
cool	normal	FALSE	P
cool	normal	TRUE	N
mild	normal	FALSE	P
mild	high	TRUE	N

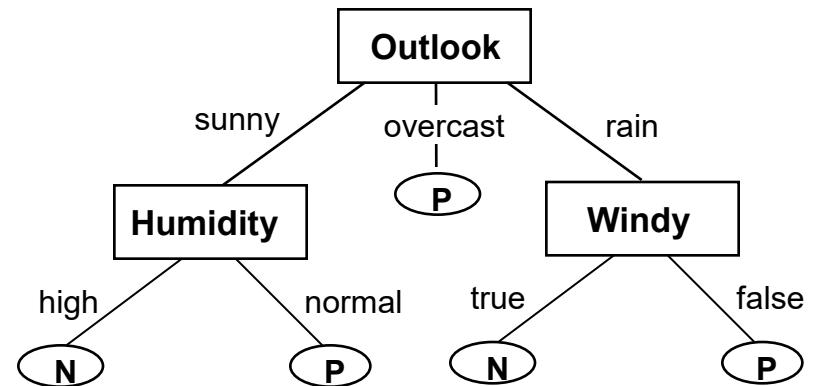


Gain(Temperature) = 0.02

Gain(Humidity) = 0.020

Gain(Windy) = 0.971

∴ Windy is chosen as the root



When to stop building the tree

- We built a decision tree by **recursively splitting our dataset**
 - Each split was performed by choosing the best feature to split (using Error rate, Gini index, or Entropy)
 - We stop when the leaf nodes is pure (i.e., all its instances have the same label)
- To avoid overfitting the stop condition can be any of the following:
 - Stop building the tree after you reach a certain depth
 - Don't split a node if the change in Error rate, Gini index, or Entropy is below some threshold
 - Don't split a node if it has less than a certain number of instances
 - Split a node only if both of the resulting leaves contain at least a certain number of instances

Decision Tree Algorithm

1. Choose the Best Feature to Split On:

- Based on an **Attribute Selection Measure (ASM)** such as **Classification error, Gini impurity, Entropy**

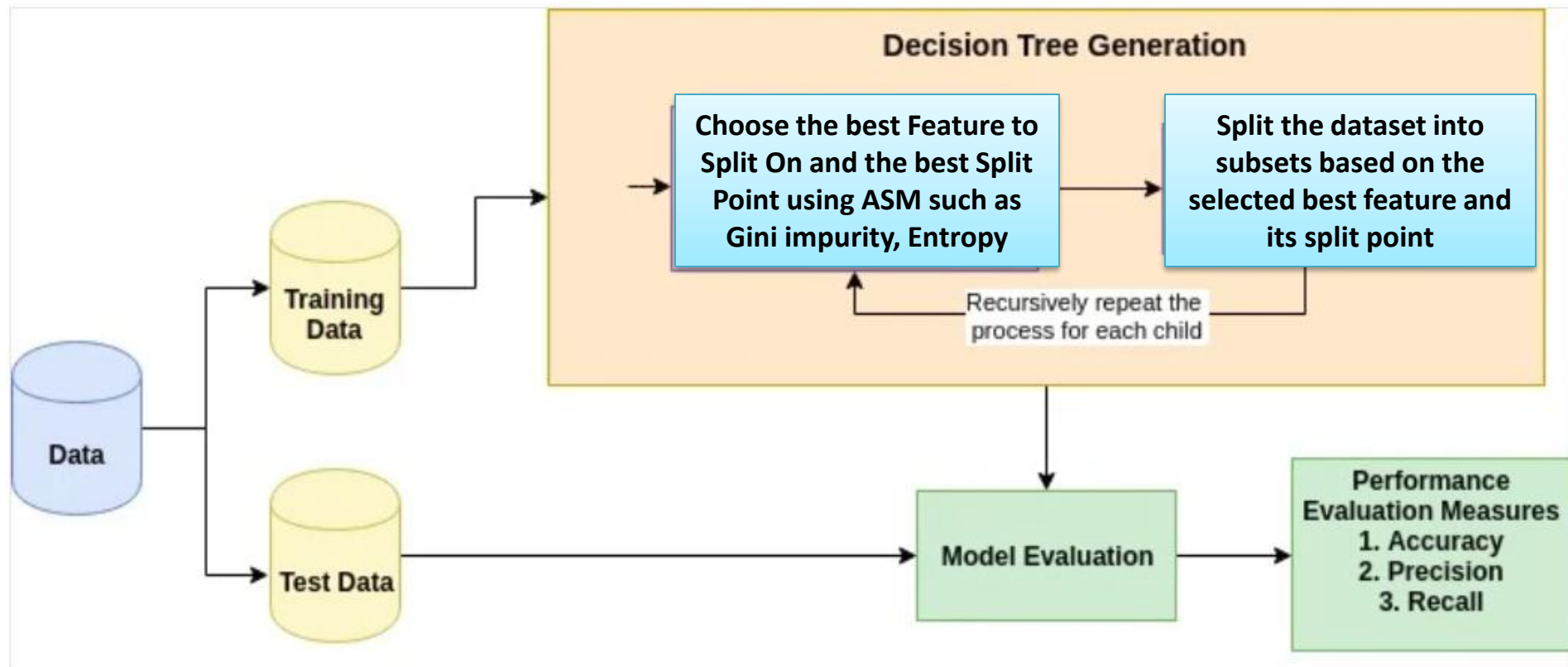
2. Determine the Split Point:

- Make the selected feature a **decision node** then find the **optimal Split Point (i.e., splitting condition/threshold)** that minimizes the impurity (using the same ASM used in step 1)

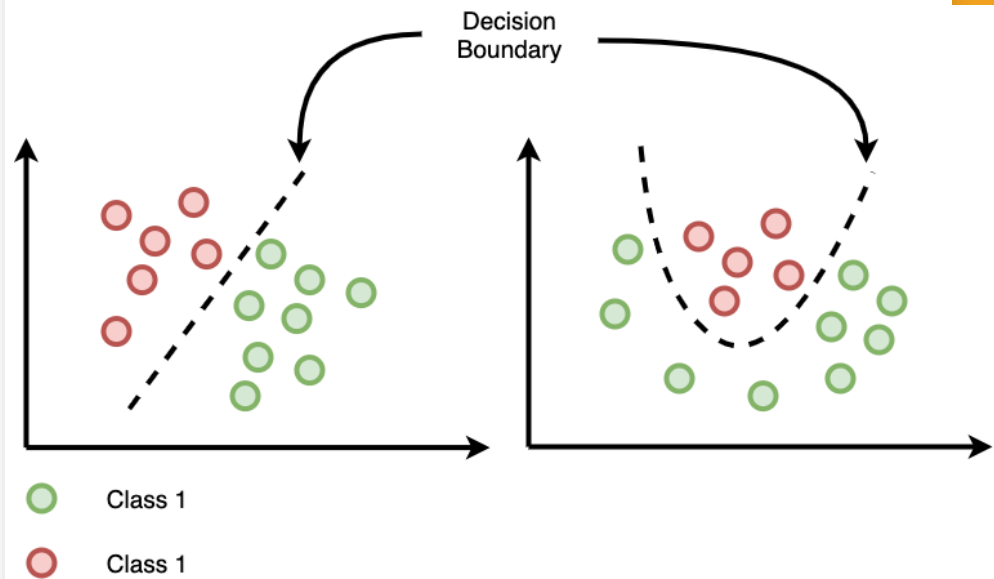
3. Recursively partition the dataset until a stop condition:

- Recursively split the dataset into subsets based on the selected best feature and its split point. Continues **until a stopping condition is met**
 - Common stopping criteria include limiting the maximum depth of the tree, minimum number of instances per leaf, or no further improvement in impurity reduction

Decision tree algorithm

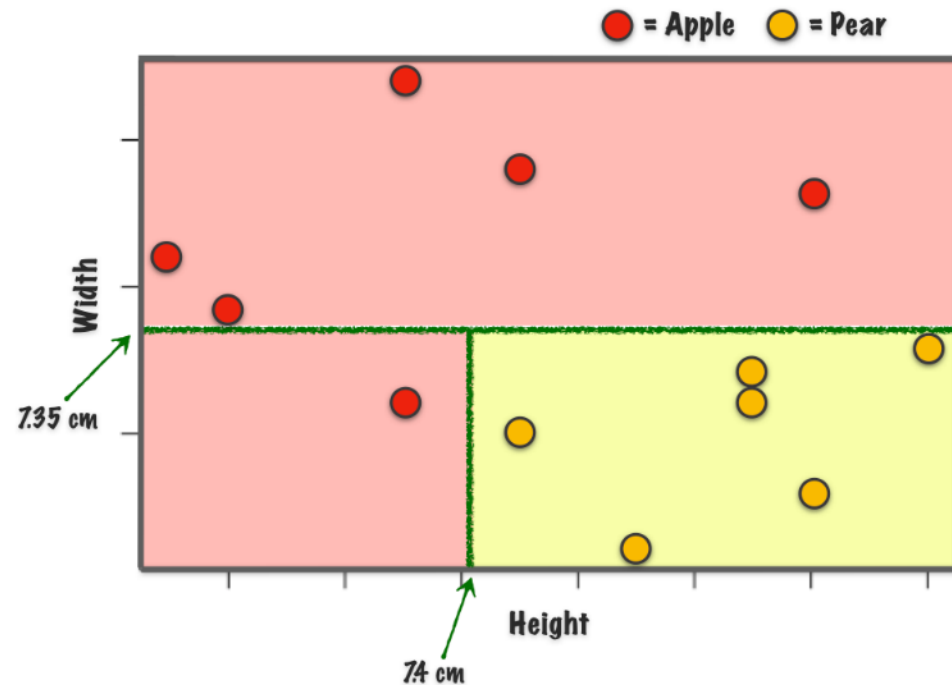
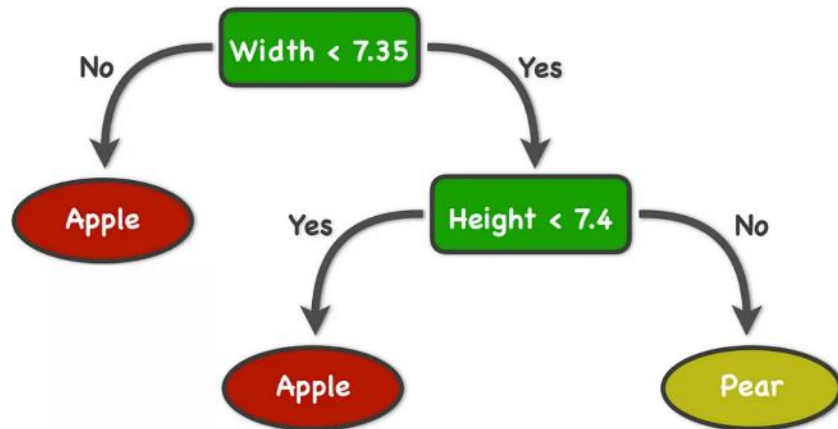


Decision Boundaries

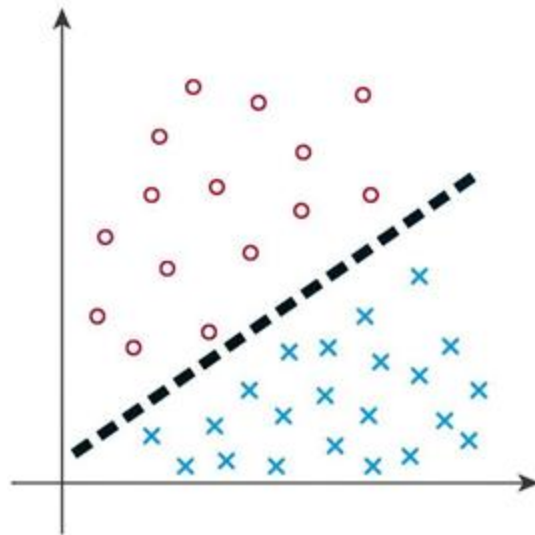


Decision tree constructs decision boundaries

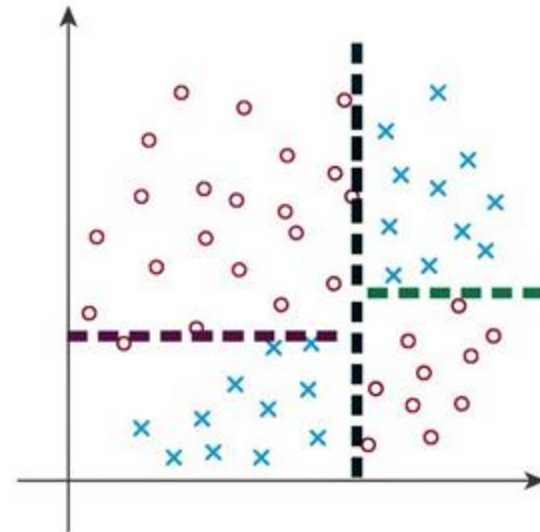
- A visual representation of a decision tree and how this translates into 2D feature space with **decision boundaries**
- The splits divide the plane into a number of boxes with each one corresponding to a classification (apple/pear) for an observation



Decision tree constructs its decision boundaries to fit the linearly inseparable dataset



Linearly separable dataset

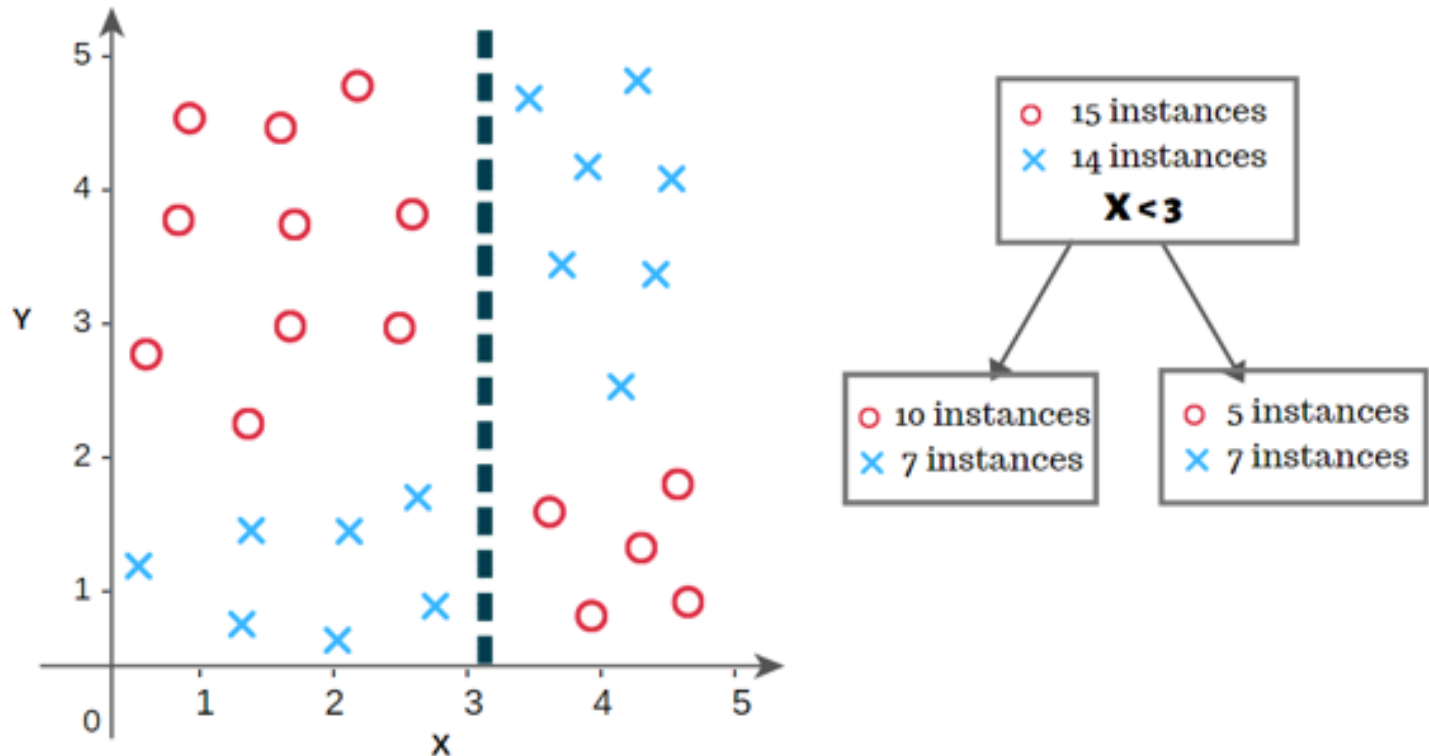


Linearly inseparable dataset

Source [link](#)

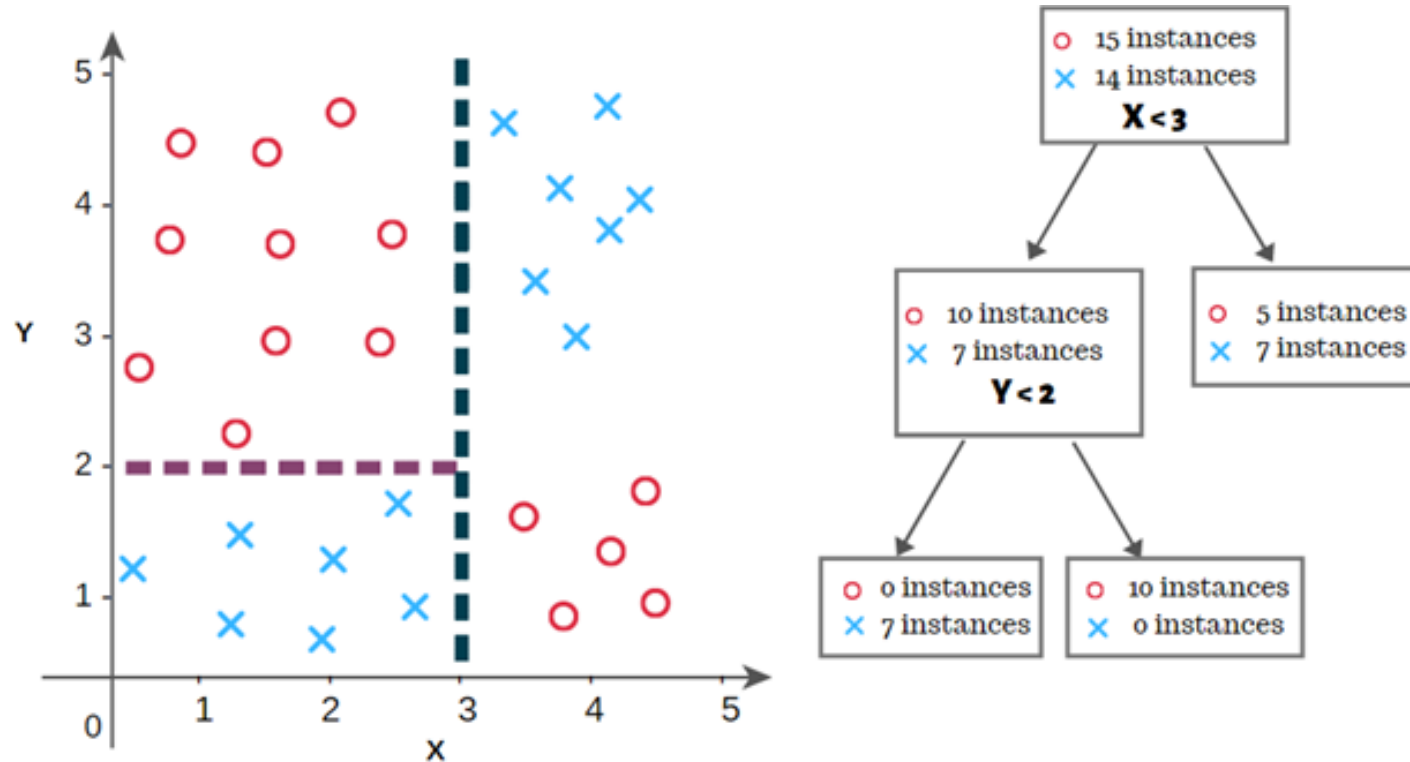
- Decision trees can, unlike linear models, fit **linearly inseparable datasets**
 - A linearly inseparable dataset is one where data points of different classes cannot be separated by a single line, as opposed to linearly separable where a single line is enough
 - DT relies on **recursive partitioning** of the datasets into “homogeneous” (pure) regions

$X < 3$ as our Split Point



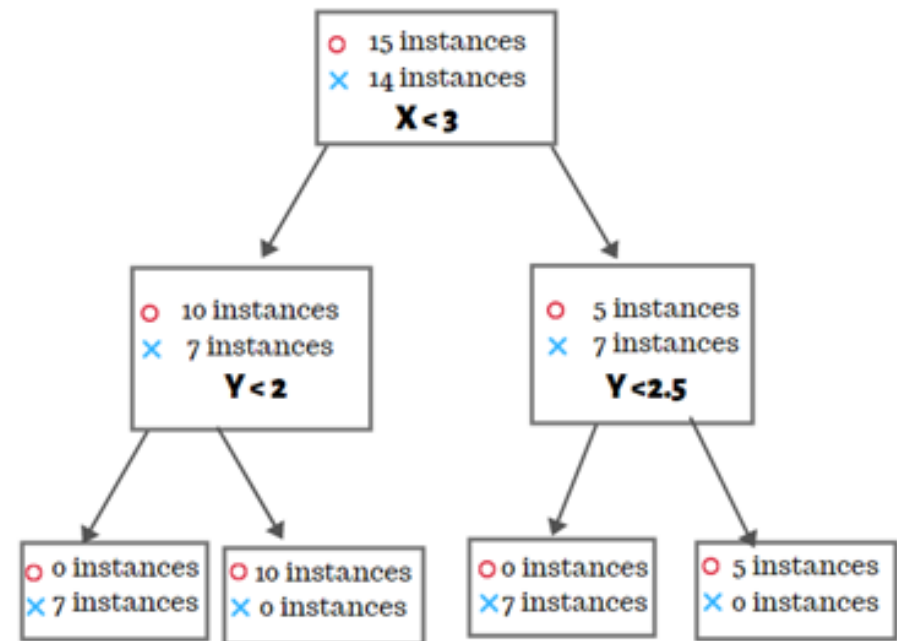
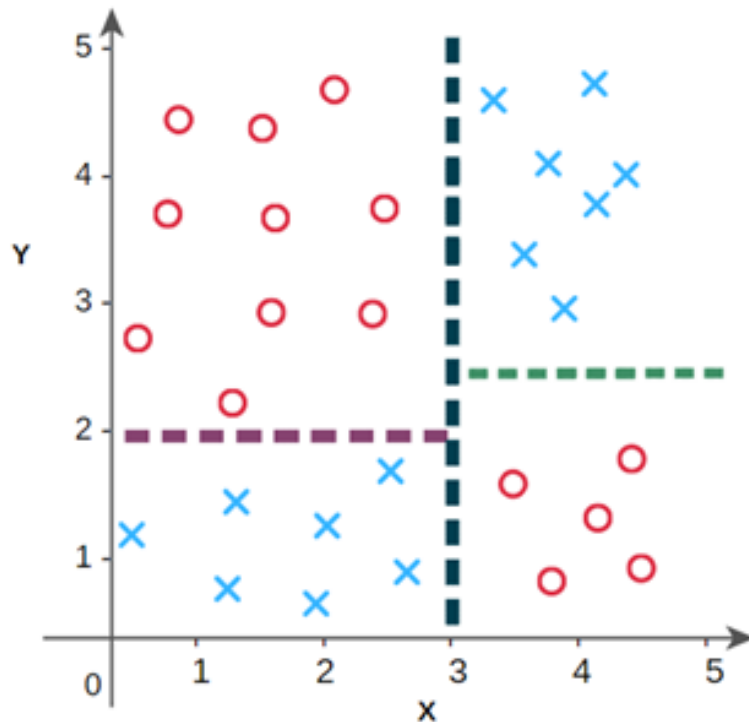
- Taking $X=3$ as our split point, yields 2 decision regions represented as child-nodes of the tree where each split contains the number of respective instances

$Y < 2$ as our Split Point for the Left region



- We'll split the left region of the resultant graph at $Y=2$
- This yields 2 decision regions represented as child-nodes of the tree. The split resulted into 2 pure leaf nodes => no further split needed for the left-side of the tree

$Y < 2.5$ as our Split Point for the Right region



- We'll split the right region of the resultant graph at $Y=2.5$
- This yields 2 decision regions represented as child-nodes of the tree. The split resulted into 2 pure leaf nodes => no further split needed for the right-side of the tree

Decision Boundaries

```
In [27]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.inspection import DecisionBoundaryDisplay

# Parameters
n_classes = 3
plot_colors = "ryb"
plot_step = 0.02

pair_of_columns = [2, 3]

# We only take the two corresponding features
X = iris.data.values[:, pair]
y = iris.target
# Train
tree_clf = DecisionTreeClassifier().fit(X, y)

plt.figure(figsize=(10,8))
plot_tree(tree_clf, filled=True)
plt.title("Decision tree trained on two attributes")
plt.show()
```

```
ax = plt.subplot(1, 1, 1)
# Plot the decision boundary
DecisionBoundaryDisplay.from_estimator(tree_clf, X, cmap=plt.cm.RdYlBu, response_method="predict",
ax=ax,
xlabel=iris.feature_names[pair[0]],
ylabel=iris.feature_names[pair[1]],
)

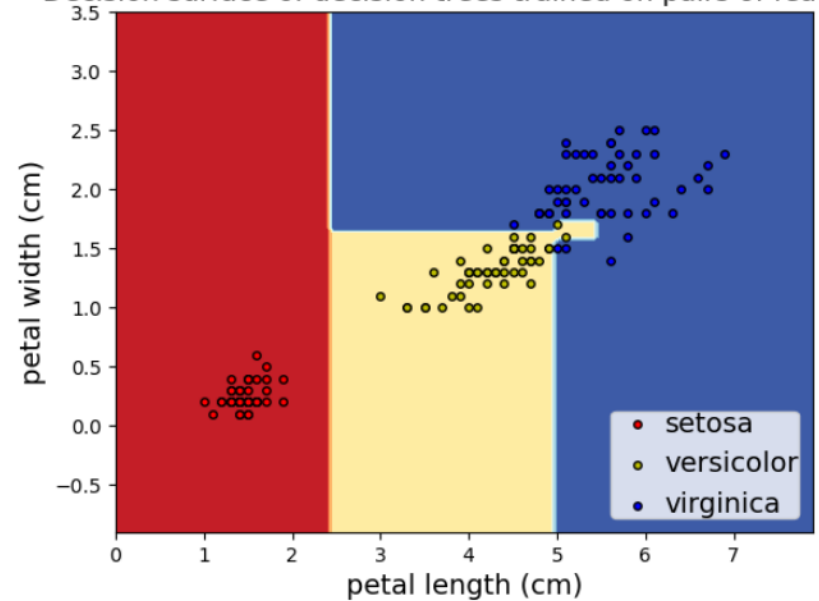
# Plot the training points
for i, color in zip(range(n_classes), plot_colors):
    idx = np.where(y == i)
    plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i],
cmap=plt.cm.RdYlBu, edgecolor="black", s=15,)

plt.title("Decision surface of decision trees trained on pairs of features")
plt.legend(loc="lower right", borderpad=0, handletextpad=0)
plt.show()
```

Decision tree trained on two attributes



Decision surface of decision trees trained on pairs of features



Summary

- A DT is a graphical representation of a set of rules for predicting or classifying new instances
- A decision tree is constructed from a set of training examples by using a tree construction algorithm such as CHAD, ID3, ...
- These algorithms differ mainly on the attribute selection measure used; GINI Index, Information gain, ...

Summary

- **Pros:**

- Resulting tree is often simple and easy to interpret
- Relatively inexpensive to construct
- More closely mirror human decision-making
- Variable selection & reduction is automatic

- **Cons:**

- The greedy approach (makes the locally optimal choice at each step) does not guarantee best solution: Since the process deals with one variable at a time, no way to capture interactions between variables
- Limited expressiveness: May not perform well where there is structure in the data that is not well captured by horizontal or vertical splits
- Non robust: sensitive to small changes in the data